2. Introduction

This chapter introduces the Nyaya Mitra system. It explains why the system is needed, what it is intended to achieve, what lies inside and outside its boundaries, and how the rest of the document is organised. The terminology used throughout the SRS is also defined here so that the technical chapters can be read without ambiguity.

2.1 Purpose of the System

Nyaya Mitra is a unified web-based legal-tech platform that brings common legal-information, document-preparation, contract, research, fraud-safety, court-support, lawyer-discovery and template workflows into one authenticated workspace.

The purpose of the system is to give an ordinary citizen, a small business owner, a law student or a legal professional a single, authenticated place in which the common first steps of a legal task can be carried out. Instead of moving between search engines, government portals, word processors, translation tools and separate fraud-checking websites, the user signs in once and chooses a module from the dashboard. A question can be asked to the legal chatbot, a contract can be reviewed for risk, a notice can be drafted, a suspicious message can be assessed, a case number can be decoded and a lawyer can be contacted, all within the same interface.

A second purpose is to return information in a form that is useful rather than merely long. Wherever possible the backend asks the language model for structured output, for example a risk score, a list of red flags, missing protections, favourable points and recommendations for a contract. Structured output allows the frontend to present the result clearly and allows the result to be stored, compared and downloaded. This design choice is visible in the contract-review, bulk-review, redline, fraud-detection and SmartModule workflows.

The third purpose is to be transparent about what the system is. Nyaya Mitra is an assistance and preparation platform. It does not replace a qualified advocate, a court, a government authority or an official legal database, and the interface and this document therefore distinguish carefully between information generated by the language model and information obtained from an official source. The purpose of this SRS is to record these intentions precisely so that the implemented system can be evaluated against them.

2.2 Scope of the System

The scope of the current release is defined by the capabilities that are present in the supplied project source. The dashboard exposes 37 modules, which are served by a mixture of dedicated pages and a shared, specification-driven module engine. The lists below state what is included in the scope of this release and what is deliberately excluded or only partially implemented, so that the document does not overstate the system.

In Scope

- 37-module dashboard.
- Registration/login/JWT protected application.
- Six original core AI flagship workflows.
- 21 generic SmartModules.
- Fraud/scam detection and community feed.
- Document upload/extraction.
- CNR tracker, lawyer directory and consultation request.
- Bulk contract review and redline DOCX.
- Contract templates, sharing and translation.

Out of Scope / Not Fully Implemented

- Legally binding e-notarization/e-signatures.
- Guaranteed live official court-status extraction where CAPTCHA/API access prevents it.
- Formal RBAC based on stored user roles.
- Native mobile apps.
- Browser phishing extension.
- Payment/subscription gateway.
- Production-grade authoritative legal RAG/database.
- Replacement of professional legal advice.

In summary, the in-scope items form a coherent, demonstrable full-stack system, while the out-of-scope items are either legally sensitive (e-notarisation, e-signatures, replacement of professional advice), dependent on external access that is not guaranteed (live court-status extraction), or infrastructure features that are better introduced in a production deployment (payments, native mobile applications, role-based access control). These items are revisited as future enhancements in Chapter 10.

2.3 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification, the present document.
AI / LLM	Artificial Intelligence / Large Language Model. Gemini 3 Flash is the LLM used by the system.
API	Application Programming Interface; the set of /api endpoints exposed by the FastAPI backend.
REST	Representational State Transfer; the style used for the HTTP endpoints.
SSE	Server-Sent Events; the one-way streaming mechanism used to deliver chatbot answers token by token.
JWT	JSON Web Token; the signed token issued at login and sent as a bearer token to protected endpoints.
CORS	Cross-Origin Resource Sharing; browser policy controlling which origins may call the API.
CNR	Case Number Record; the unique identifier of a case in the Indian e-Courts system.

Term	Meaning
RBAC	Role-Based Access Control; a future enhancement, not stored in the current user model.
RAG	Retrieval-Augmented Generation; a future approach for grounding legal research on authoritative sources.
RTI	Right to Information; one of the Core Legal Services modules.
DOCX	Office Open XML word-processing format generated and read by python-docx.
SmartModule	A generic module defined by an entry in the backend MODULE_SPECS registry and rendered by a shared frontend form.
DFD	Data Flow Diagram, drawn at context (level 0), level 1 and level 2.
ER / ERD	Entity-Relationship (Diagram) describing the MongoDB data model.
UML	Unified Modeling Language, used for class, sequence and activity diagrams.

2.4 Organisation of this Document

The document follows the institutional index and is organised in eleven chapters. The table below gives a short roadmap so that a reader can move directly to the part that is relevant to a particular question.

Chapter	Title	Content
1	About Department	Department background, project profile and alignment with departmental objectives.
2	Introduction	Purpose and scope of the system, definitions and document organisation.
3	System Description	Problem definition, existing system and proposed system.
4	Users of the System	User categories, access characteristics and user-to-module mapping.
5	System Specifications	Hardware requirements, software requirements, tools and technology.
6	Requirements	Functional and non-functional requirements.
7	Use Cases	Use case list, scenarios, specifications and traceability.
8	System Design (Diagrams)	Architecture, use case diagrams, DFDs, UML diagrams, ER diagram, data model, API and interface design.
9	Spiral Model Iterations	Planning and risk analysis, requirements and design refinement, validation.
10	Conclusion and Future Enhancements	Summary of results and the roadmap of enhancements.
11	Bibliography	Technical references and sources.

3. System Description

This chapter describes the system from the point of view of the problem that it addresses. It first defines the problem, then analyses the existing way of working, and finally presents the proposed system together with its modules and workflows.

3.1 Problem Definition

Access to legal help in India is uneven. A person who receives a rental agreement, an employment contract, a cheque-bounce notice or a suspicious bank message must first understand what the document says, what risks it carries and what action is appropriate. Professional advice is the most reliable route, but it is not always immediately available, and a person often needs a quick first-pass understanding before deciding whether and whom to consult. At the same time, the information needed for that first pass is scattered across many websites, government portals and documents written in technical language.

The difficulty is therefore not the lack of information but its fragmentation and the effort needed to interpret it. A user must search, read, compare, draft and verify, usually using several unrelated tools. The nine problem statements recorded below describe the specific needs that the Nyaya Mitra system sets out to meet.

3.1.1 Problem Statements

1. Users need an accessible first point of contact for general legal information.
2. Users need faster first-pass understanding of legal documents and contracts.
3. Non-lawyers need assistance preparing structured petitions and notices.
4. Users need quick risk assessment of suspicious messages, emails and URLs.
5. Legal research needs an intelligent search and summarisation layer.
6. Case/court utilities are fragmented from document and legal-preparation workflows.
7. Comparing several contracts manually is time-consuming.
8. Users need reusable Indian contract templates with generated text and DOCX output.
9. Regional-language support is needed for wider accessibility.

3.1.2 Need for the New System

The following needs follow directly from the problem statements. Each need is mapped to one or more modules in the proposed system.

- Unify legal-preparation workflows in one workspace.
- Reduce first-pass document review and drafting time.
- Return structured AI results rather than only free-form text.
- Provide proactive scam/fraud risk assessment.
- Support document uploads and extraction.

- Enable bulk contract comparison and redline downloads.
- Connect legal preparation with lawyer discovery.
- Provide multilingual template drafting and translation.

Taken together, these needs justify a unified platform. A single workspace removes the cost of switching between tools, and the use of structured AI output makes the results easier to read, store and reuse than free-form text.

3.2 Existing System / Work Environment

Users commonly rely on separate lawyers, search engines, legal portals, document editors, government websites and cyber-fraud resources. Routine activities such as understanding a contract, preparing a petition, checking a suspicious message or finding a lawyer therefore require multiple disconnected tools.

- Legal information is distributed across many sources.
- Document and contract review can be slow without structured assistance.
- Drafting legal notices and petitions is difficult for non-lawyers.
- Scam messages, emails and URLs require separate safety checks.
- Court-related information may require separate official portal visits.
- Template creation, translation and document download are often separate workflows.
- Lawyer discovery is not always connected to legal preparation.

Consider a typical journey in the existing work environment. A small business owner receives a service agreement by email. To understand it, the owner reads the clauses, searches the internet for unfamiliar terms, perhaps asks an acquaintance, and eventually pays for a lawyer's opinion. If the agreement needs changes, a separate word processor is used to prepare a revised version. If the owner later receives a suspicious payment message, a different website or helpline is needed to check it. If a dispute reaches a court, the case number must be decoded and searched on an official portal, which may require CAPTCHA entry and a clear idea of what the stages mean.

Each step in this journey is individually manageable, but the absence of a connecting workflow makes the whole process slow and error-prone. No tool in the existing environment keeps the history of what the user analysed, produces a downloadable redline of a contract, translates a template into a regional language or links the preparation work with a directory of lawyers. These gaps define the opportunity for the proposed system.

3.3 Proposed System

The proposed system, Nyaya Mitra, is a single authenticated web application that brings these workflows together. A user registers or logs in, lands on a dashboard that lists all 37 modules in three categories, and opens the desired module. Behind the interface, a FastAPI backend validates the request, calls the language

model or the document-processing libraries as required, stores the result in MongoDB and returns a structured response to the React frontend.

3.3.1 System Overview

- React frontend with public pages, protected /app routes and dedicated module pages.
- FastAPI backend with REST APIs and Server-Sent Events for chatbot streaming.
- MongoDB persistence through Motor.
- JWT authentication and bcrypt password hashing.
- Gemini 3 Flash for chat, analysis, drafting, review, research, fraud assessment, generic modules, template generation and translation.
- PDF/DOCX/TXT extraction using pypdf and python-docx.
- Bulk contract processing with concurrent AI reviews.
- DOCX redline generation.
- Lawyer directory, filtering and consultation-request workflow.
- CNR tracking assistance with official e-Courts deep linking.
- Contract template generation, DOCX downloads, public sharing and translation.

The technology choices listed above are deliberately mainstream and open-source wherever possible. This keeps the development cost low, makes the code easy to maintain and allows each layer to be replaced or scaled independently. Importantly, the browser never calls the language model directly; all AI requests pass through the backend, which can authenticate the user, validate the input and keep the provider credentials private.

3.3.2 Current Module Count

Category	Count	Current Scope
Core Legal Services	19	Case, consultation, court, document, contract, research, cyber, IP, grievance, compliance, evidence, dispute, risk, traffic, RTI, notary and analytics workflows.
Advanced AI Features	11	Research assistant, drafting, contract agent, bulk review, templates, CRM, case agent, voice, multilingual, mock court and lawyer marketplace.
AI Scam&Fraud Detection	7	Message, email, URL, risk scoring, reporting, community alerts and awareness.
Total	37	Current dashboard registry returned by /api/modules.

The registry returned by the /api/modules endpoint contains 37 modules: 19 Core Legal Services, 11 Advanced AI Features and 7 AI Scam and Fraud Detection modules. Two later additions, Bulk Contract Review and the Contract Templates Library, are part of the Advanced AI category. A complete inventory of the modules is given in Section 9.3.3.

3.3.3 Core Flagship Modules

Module	Implemented Behaviour
AI Legal Chatbot	Streaming SSE conversation with session persistence.
AI Document Analyzer	Structured document analysis; pasted text and supported file uploads.
Petition&Notice Drafting	Formal draft from case type, parties, facts, relief and jurisdiction.
AI Contract Review	Risk score/findings, red flags, missing protections, favourable points and recommendations.
Legal Research Engine	AI-assisted research results with legal authorities/case information.
AI Scam/Fraud Detection	Message/email/URL risk assessment, scan history and community alerts.

Six workflows form the original flagship set and have dedicated pages with specialised behaviour: the AI Legal Chatbot with streaming responses, the AI Document Analyzer, Petition and Notice Drafting, AI Contract Review, the Legal Research Engine and AI Scam and Fraud Detection. The remaining modules are served by the common SmartModule engine, which allows new modules to be added by registering a specification rather than writing a new page.

3.3.4 Additional Implemented Workflows

Workflow	Current Capability
File Upload	PDF/DOCX/TXT extraction for analysis/review.
CNR Tracker	CNR validation/decoding, portal reachability, official deep link and AI-inferred stage/next steps.
Lawyer Directory	Seeded records, filters, sorting, profile view and consultation request.

Bulk Contract Review	2– 20 files, concurrent review, comparative statistics and league table.
Redline DOCX	Downloadable annotated contract with risk summary and recommendations.
Templates	12 Indian-oriented templates, AI generation, DOCX and public sharing.
Template Translation	English + Hindi, Tamil, Telugu, Bengali and Marathi.
Voice Assistant	Text-driven spoken-style legal answer; browser voice input is not currently implemented.

The workflows above were added after the original 35-module concept and make the platform considerably more practical. File upload removes the need to copy text manually, the CNR tracker and lawyer directory connect preparation with court and professional resources, bulk review and redline output support contract negotiation, and the template library with translation extends the platform to users who prefer a regional language.

3.3.5 Comparison of the Existing and the Proposed System

Aspect	Existing work environment	Proposed system (Nyaya Mitra)
Entry point	Separate search engines, portals and offices	Single authenticated dashboard with 37 modules
Document understanding	Manual reading; paid opinion needed	Structured AI analysis of pasted text or PDF/DOCX/TXT upload
Contract review	Slow, clause-by-clause manual work	Risk score, red flags, missing protections and recommendations; bulk review of 2-20 files
Contract markup	Manual edits in a word processor	Downloadable redline DOCX with annotations
Drafting	Requires legal drafting skill	Petition and notice drafts generated from case type, parties, facts, relief and jurisdiction
Fraud checking	Separate websites or helplines	Message, email and URL assessment with scan history and community alerts
Court information	Official portal visit; CAPTCHA	CNR decoding, reachability check, official deep link and AI-inferred stage
Language support	Mostly English	Template translation into Hindi, Tamil, Telugu, Bengali and Marathi
Record keeping	Scattered files and browser history	User-scoped histories stored in MongoDB

The comparison shows that the proposed system does not attempt to replace the existing sources of authority. It reduces the effort needed to reach them by providing a structured first pass, and it clearly labels AI-generated content so that the user knows when verification is needed.

4. Users of the System

This chapter identifies the people who use Nyaya Mitra and describes how each group interacts with it. Because the current implementation uses a single authenticated account type, the chapter is careful to separate what is implemented from what is planned.

4.1 Categories of Users

User Type	Current Support
Authenticated General User	Core legal, document, fraud, template, court and lawyer workflows.
Lawyer / Legal Professional	Research, review, drafting, case utilities, client-oriented workflows and marketplace-related functions.
Law Student	Research, mock court, multilingual and learning-oriented workflows.
Administrator	No separate stored role in current auth; future RBAC enhancement.

The authenticated general user is the primary actor of the system. This user may be a citizen with a personal legal question, a tenant or employee reviewing an agreement, a small business owner who needs a template or a contract review, or a person who has received a suspicious message. For this group the most relevant modules are the chatbot, document analyzer, contract review, templates, fraud detection, CNR tracker and lawyer directory.

A lawyer or legal professional can use the same account to benefit from the research engine, contract and bulk review, redline generation and drafting support, as well as the client-oriented and marketplace-related modules. For this group the system acts as a productivity aid that produces a structured first draft or review which the professional then verifies and refines using professional judgement.

A law student is supported through research, mock court, multilingual and learning-oriented workflows. The student can practise argumentation in the Mock Court module, study legal topics through the research engine and obtain explanations in more than one language. An administrator role is not stored in the current authentication model; administrative functions such as user management and moderation are listed as a future enhancement.

4.2 Access and Authentication Characteristics

All users follow the same access path. A new user registers with a name, email address and password; an existing user logs in with email and password. The backend enforces a minimum password length of six characters, stores only a bcrypt hash and returns a signed JWT. The frontend keeps the token and sends it as a bearer token on every protected request. Public pages are limited to the landing page, the authentication

page and the public template share view at /share/:id; all other routes are under /app and require authentication.

Because the registration model does not store a role, every authenticated user receives the same set of permissions. The distinction between a general user, a lawyer and a law student therefore exists at the level of the workflows that each person chooses to use rather than at the level of enforced authorisation. Records created by a user, such as chat history, analyses, drafts, scans and module runs, are scoped to that user's identity so that one user cannot read another user's history. Introducing formal roles is the first item in the future-enhancement roadmap.

4.3 User-to-Module Interaction Summary

User type	Typical goal	Principal modules
Authenticated General User	Understand a document, prepare a notice, check a scam, find a lawyer	Chatbot, Document Analyzer, Petition Drafting, Contract Review, Fraud Detection, CNR Tracker, Lawyer Directory, Templates
Lawyer / Legal Professional	Speed up research, review and drafting; manage client work	Legal Research, Contract Review, Bulk Review, Redline DOCX, Drafting, Legal Client CRM, Lawyer Marketplace
Law Student	Learn, practise and study in more than one language	Legal Research, Mock Court, Multi-Language Support, Legal Chatbot
Administrator	Manage users and content (planned)	Not available in the current release; future RBAC enhancement

The table shows that most modules are useful to more than one group. This overlap is the reason the system uses a single account type and a shared dashboard: the same Contract Review module serves a tenant and a lawyer equally well, and the difference lies in how the output is used rather than in how it is produced.

5. System Specifications

This chapter lists the hardware and software environment in which Nyaya Mitra is developed, deployed and used. The specification is deliberately modest: the heavy computation is performed by the external language-model service, so neither the server nor the client requires specialised hardware.

5.1 Hardware Requirements

The hardware requirements apply to the machine that runs the backend, the database and the frontend build during development or local deployment. A client user needs only a device with a current desktop web browser and an internet connection.

Component	Specification
Processor	Intel Core i5 / Ryzen 5 or higher
RAM	8 GB minimum; 16 GB recommended
Storage	20 GB free or more
Network	Required for AI calls and external portal checks
Client device	Any computer with a current Chrome, Edge or Firefox browser; responsive layouts are provided

The 8 GB minimum allows the React development server, the FastAPI server and a local MongoDB instance to run together. The 16 GB recommendation provides headroom for running the end-to-end test tooling and the pytest suites at the same time. Storage is needed mainly for the dependency trees of the two runtimes, the database files and uploaded documents. Because every AI workflow depends on the network, a stable connection is the most important non-hardware requirement.

5.2 Software Requirements

Category	Specification
Operating system	Windows 10/11, macOS or Linux
Runtime	Python 3.10+ for the backend; Node.js with an npm-compatible package manager for the frontend
Browser	Current Chrome, Edge or Firefox
Database	MongoDB, local or hosted, accessed through the Motor asynchronous driver
Backend frameworks and libraries	FastAPI, Pydantic, PyJWT, bcrypt/passlib, pypdf, python-docx, emergentintegrations
Frontend frameworks and libraries	React 19 (Create React App), React Router 7, Tailwind CSS with custom CSS, Axios
AI service	Gemini 3 Flash through the emergentintegrations library
Testing tools	pytest for the backend and end-to-end test tooling for the user interface
Version control	Git / GitHub

The software stack is cross-platform. Python and Node.js are available on all three supported operating systems, and MongoDB can be installed locally or consumed as a hosted service, so the project can be

demonstrated on a student laptop and later moved to a cloud environment without changing the code. The pinned dependency list lives in the backend requirements file and the frontend package manifest of the project archive.

5.3 Tools and Technology

The table below lists the technologies used in each layer and the purpose for which each is used. The choices follow the usual pattern for modern full-stack applications: a component-based single-page frontend, an asynchronous Python API and a document database whose flexible schema suits the variety of structured AI results that the system stores.

Layer	Technology	Purpose
Frontend	React 19 / Create React App	UI
Routing	React Router 7	Navigation
Styling	Tailwind CSS + custom CSS	Design system
HTTP	Axios	API communication
Backend	FastAPI	API/application layer
Database	MongoDB + Motor	Persistence
Authentication	PyJWT + bcrypt/passlib	JWT/password security
AI	Gemini 3 Flash + emergentintegrations	AI workflows
Documents	pypdf + python-docx	Extraction/generation
Testing	pytest + E2E tooling	Validation
Version Control	Git/GitHub	Source control

React was chosen for its component model and large ecosystem, and React Router provides the nested routing needed for the public and protected areas. FastAPI offers automatic request validation through Pydantic, native asynchronous support (which the bulk-review workflow relies on) and straightforward streaming responses for Server-Sent Events. MongoDB stores JSON-like documents, which matches the structured results returned by the language model. PyJWT and bcrypt provide token-based authentication and secure password hashing, while pypdf and python-docx handle the extraction and generation of documents.

6. Requirements

This chapter states what the system must do and how well it must do it. The functional requirements describe the behaviour of the 22 principal functions, and the non-functional requirements describe the quality attributes that apply across all of them. The requirements are written so that each can be verified by a test, and they are traced to use cases in Section 7.3 and to test cases in Section 9.4.4.

6.1 Functional Requirements

The functional requirements are listed in the order in which a user typically meets the functions: authentication first, followed by the AI workflows, the fraud suite, the generic module engine, the court and lawyer utilities and finally the template features.

ID	Function	Requirement
FR-01	Registration	Create account with name, email and password.
FR-02	Login	Authenticate and issue JWT.
FR-03	Protected Access	Protect application/API operations using bearer JWT.
FR-04	Chat	Stream legal answers using SSE.
FR-05	Chat Persistence	Store messages and retrieve sessions.
FR-06	Document Analysis	Analyse pasted document text.
FR-07	File Analysis	Extract supported PDF/DOCX/TXT uploads.
FR-08	Drafting	Generate petition/notice drafts.
FR-09	Contract Review	Return structured contract risk analysis.
FR-10	Research	Generate AI-assisted legal research results.
FR-11	Fraud	Analyse message/email/URL content.
FR-12	Community Alerts	Store/display suspicious community alerts.
FR-13	SmartModules	Run 21 generic modules through common spec/run APIs.
FR-14	CNR	Provide CNR assistance and official portal link.
FR-15	Lawyers	Filter/search lawyers and request

		consultation.
FR-16	Bulk Review	Review 2– 20 contracts comparatively.
FR-17	Redline	Generate downloadable redline DOCX.
FR-18	Templates	List/generate 12 templates.
FR-19	Template DOCX	Generate downloadable DOCX.
FR-20	Sharing	Create/view public template shares.
FR-21	Translation	Translate templates into five Indian languages.
FR-22	Voice	Generate language-specific spoken-style legal responses.

Requirements FR-01 to FR-03 establish the identity and protection of the user. Every other function is protected by the bearer-token mechanism stated in FR-03, so no AI or data workflow can be reached without authentication. Requirements FR-04 and FR-05 cover the chatbot, which must stream its answers and must persist the conversation so that sessions can be revisited.

Requirements FR-06 to FR-10 cover the document-centred workflows: analysis of pasted or uploaded text, drafting, contract review and research. Requirements FR-11 and FR-12 define the fraud suite and the community alert feed, FR-13 the SmartModule engine, and FR-14 and FR-15 the CNR and lawyer utilities. Requirements FR-16 and FR-17 extend contract review to bulk comparison and to redline output, and FR-18 to FR-21 cover the template library, DOCX output, public sharing and translation. FR-22 describes the voice-style assistant, whose input is currently text-based.

6.2 Non-Functional Requirements

The non-functional requirements apply to the whole system. They are expressed in measurable or observable terms wherever the implementation allows, and they are the basis of the security and validation activities described in Chapter 9.

Area	Requirement
Security	Passwords hashed; protected APIs validate JWT.
Privacy	User histories scoped to authenticated identity.
Usability	Major workflows accessible from dashboard.

Maintainability	Shared SmartModule architecture reduces duplication.
Reliability	Invalid input/files/auth failures handled with controlled errors.
Performance	Streaming chat and concurrent bulk review.
Scalability	Module registry allows future modules/integrations.
Compatibility	Current desktop browsers and responsive layouts.
AI Transparency	AI-inferred court information must be distinguished from official data.

Security and privacy requirements are the most important in this domain because users may paste personal agreements and sensitive facts. Passwords are never stored in plain text, protected APIs validate the JWT, and histories are scoped to the authenticated identity. Reliability requires that invalid input, invalid files and authentication failures produce controlled errors instead of server faults, and performance is addressed by streaming chat output and by running bulk reviews concurrently.

The remaining requirements concern maintainability and honesty. Maintainability is served by the shared SmartModule architecture, which means that a new module is added by registering a specification instead of creating a new page, and scalability follows from the same registry design. AI transparency is a distinctive requirement of this project: any court information that is inferred by the model must be clearly distinguished from data obtained from the official portal.

6.3 Assumptions and Constraints

The requirements above rest on a small number of assumptions. It is assumed that the user has a stable internet connection, that the Gemini service is reachable and responds within a reasonable time, that users access the system from a current desktop or responsive mobile browser, and that uploaded documents are text-based PDF, DOCX or TXT files of a size accepted by the validation rules. If one of these assumptions fails, for example when the language model is unavailable, the system is expected to return a controlled error message rather than an unhandled failure.

Several constraints also shape the design. The current registration model stores no formal role, so authorisation is limited to authenticated versus unauthenticated access. Court-status information cannot be guaranteed from the official portal because of CAPTCHA and API restrictions, so the CNR tracker provides assistance and a deep link instead of live scraping. Lawyer records are seeded project data rather than professionally verified profiles, and no payment gateway is implemented. These constraints are repeated in the limitations listed in Section 9.4.6 so that the evaluation of the system is not misled.

7. Use Cases

A use case describes a goal that an actor achieves by interacting with the system. This chapter lists the 17 use cases of Nyaya Mitra, describes the principal scenarios in detail, provides a specification summary for every use case and finally traces the functional requirements to the use cases that realise them. The corresponding use case diagrams appear in Section 8.2.

7.1 Use Case List

All current use cases are performed by an authenticated user, with the exception of registration and login, which are performed by a visitor who has not yet been authenticated. Because no administrator role is stored, no administrator use case is listed.

ID	Use Case	Actor	Description
UC-01	Register	User	Create account.
UC-02	Login	User	Authenticate and receive JWT.
UC-03	Ask Legal Chatbot	Authenticated User	Send question and receive streamed response.
UC-04	Analyse Document	Authenticated User	Paste/upload document and receive structured analysis.
UC-05	Draft Petition/Notice	Authenticated User	Generate formal draft from supplied facts.
UC-06	Review Contract	Authenticated User	Analyse contract risk.
UC-07	Bulk Review	Authenticated User	Compare 2– 20 contracts.
UC-08	Generate Redline	Authenticated User	Create annotated DOCX.
UC-09	Legal Research	Authenticated User	Search legal issue with AI assistance.
UC-10	Scan Scam	Authenticated User	Assess message/email/URL risk.
UC-11	Track CNR	Authenticated User	Decode/check CNR and open official portal.
UC-12	Find Lawyer	Authenticated User	Filter lawyers and request consultation.
UC-13	Generate Template	Authenticated User	Generate selected contract template.

UC-14	Translate Template	Authenticated User	Translate template to supported Indian language.
UC-15	Share Template	Authenticated User	Create public share view.
UC-16	SmartModule	Authenticated User	Run a generic legal module.
UC-17	Mock/Voice/Multilang	Authenticated User	Use learning, voice-style or translation module.

The use cases cover the full breadth of the system. UC-01 and UC-02 provide access, UC-03 to UC-09 form the legal-assistance and document workflows, UC-10 is the fraud-detection workflow, UC-11 and UC-12 are the court and lawyer utilities, UC-13 to UC-15 are the template workflows, and UC-16 and UC-17 represent the generic and specialised module engines.

7.2 Use Case Descriptions

The six scenarios below describe the most important workflows step by step. Each scenario shows the sequence of interactions between the user, the frontend, the backend and the supporting services, and is realised in the sequence diagrams of Section 8.4.2.

7.2.1 Registration and Login

Registration and login are the entry points to the system. The scenario below applies to both: in registration the backend creates a new user record, and in login it verifies the supplied credentials against the stored hash.

10. User opens authentication.
11. Registration accepts name/email/password.
12. Backend validates and hashes password.
13. JWT is issued.
14. Frontend redirects to /app.

On success the user never sees the token, but all later requests carry it. On failure, for example a wrong password, the backend returns a controlled authentication error and the frontend remains on the authentication page.

7.2.2 AI Legal Chatbot

The chatbot answers general legal questions in a conversational style. Because language-model answers can take several seconds to complete, the backend streams them to the browser using Server-Sent Events so that the user can start reading immediately.

15. User enters question.

16. Frontend calls /api/chat/stream.
17. Backend streams Gemini output using SSE.
18. Messages are persisted.
19. History can be retrieved.

Each conversation has a session identifier. When the user returns, the frontend can list recent sessions and load the stored messages, so the dialogue can be continued where it stopped.

7.2.3 Document Analysis

Document analysis helps a user understand a legal document quickly. The user may paste text or upload a PDF, DOCX or TXT file, and the system produces a structured analysis containing key findings, obligations, risks and missing items.

20. User pastes text or uploads file.
21. Backend extracts file text.
22. AI analysis prompt is executed.
23. Structured result is stored and displayed.

File extraction is performed in the backend: pypdf reads the pages of a PDF, and python-docx reads the paragraphs and tables of a DOCX file. An invalid or oversized file produces a controlled validation error and no AI call is made.

7.2.4 Contract Redline

The redline scenario builds on contract review. After the AI review has produced structured findings, the user can request an annotated Word document in which risky clauses are highlighted and explained.

24. User submits contract.
25. AI review produces structured findings.
26. Redline endpoint creates DOCX.
27. User downloads annotated document.

The redline endpoint supports three starting points: submitted text, a cached review that was produced earlier, and an uploaded file. The generated document contains a cover section, an executive summary, the contract text, risk annotations, missing protections, favourable points and recommendations.

7.2.5 Fraud Detection

Fraud detection assesses whether a message, email or link is likely to be a scam. The user pastes the suspicious content and selects the content type or leaves it on automatic detection.

28. User submits suspicious content.
29. AI assesses risk.

30. Scan is stored.
31. Suspicious/dangerous results can create community alerts.

When a scan is rated suspicious or dangerous, a masked version is added to the community alert feed so that other users can be warned. The masking is intended to avoid exposing personal details from the original message.

7.2.6 Template Translation

Template translation lets a user obtain a contract template in a regional language. The text is first generated in English and is then translated on request into Hindi, Tamil, Telugu, Bengali or Marathi.

32. User generates template.
33. User selects language.
34. AI translates while preserving protected fields.
35. Translated text can be downloaded/shared.

Protected fields such as names, numbers and dates are preserved during translation so that the legal substance of the document is not altered. Translated versions are cached, and the translated text can be downloaded as a DOCX file or shared through a public link.

7.2.7 Use Case Specification Summary

The table below summarises the precondition, main flow and postcondition of every use case, including those that are not described as separate scenarios above.

ID	Precondition	Main flow	Postcondition
UC-01	Visitor has no account	Submit name, email and password; backend validates, hashes the password and stores the user	Account created and JWT issued
UC-02	Account exists	Submit email and password; backend verifies the hash	JWT issued; user enters /app
UC-03	User authenticated	Enter a question; answer streams through SSE; messages are saved	Session history available
UC-04	User authenticated	Paste text or upload PDF/DOCX/TXT; text extracted; AI analysis run	Structured analysis stored and shown
UC-05	User authenticated	Enter case type, parties, facts, relief and jurisdiction; AI drafts the document	Draft stored and displayed
UC-06	User authenticated	Submit contract text or file; AI review produces findings	Risk score, red flags, missing protections, favourable points and recommendations stored
UC-07	User authenticated; 2-20 files	Files are reviewed concurrently; per-file failures are isolated	Batch statistics, safest/riskiest callouts and league table
UC-08	Contract text, cached review or file available	Redline endpoint builds an annotated document	Redline DOCX downloaded

ID	Precondition	Main flow	Postcondition
UC-09	User authenticated	Enter a legal issue; research prompt executed	Structured research result stored and shown
UC-10	User authenticated	Submit message, email or URL; AI assesses risk; scan saved	Risk level and guidance shown; community alert if suspicious or dangerous
UC-11	User authenticated	Enter CNR; length validated and components decoded; portal reachability checked	Official deep link and AI-inferred stage shown
UC-12	User authenticated	Filter and sort lawyers; open a profile; submit a consultation request	Consultation request stored; no payment
UC-13	User authenticated	Choose one of 12 templates; fill typed fields; AI generates text	Template text generated; DOCX available
UC-14	Template generated	Choose a target language; AI translates and preserves protected fields	Translated text shown and downloadable
UC-15	Template generated	Request a share; body and title are stored	Public link /share/:id created
UC-16	User authenticated	Fetch module specification; render dynamic form; submit to /api/module/run	Structured result displayed; run stored
UC-17	User authenticated	Submit input to Mock Court, Voice or Multi-Language module	Learning, spoken-style or translated response returned

7.3 Requirement-to-Use Case Traceability

Traceability shows that every functional requirement is realised by at least one use case and that no use case exists without a supporting requirement. The mapping also helps the tester choose which use case scenario to run when verifying a given requirement.

Requirement	Function	Realised by use case
FR-01	Registration	UC-01
FR-02	Login	UC-02
FR-03	Protected access	UC-02 to UC-17
FR-04	Chat streaming	UC-03
FR-05	Chat persistence	UC-03
FR-06	Document analysis	UC-04
FR-07	File analysis	UC-04
FR-08	Drafting	UC-05
FR-09	Contract review	UC-06
FR-10	Research	UC-09
FR-11	Fraud analysis	UC-10
FR-12	Community alerts	UC-10
FR-13	SmartModules	UC-16

Requirement	Function	Realised by use case
FR-14	CNR assistance	UC-11
FR-15	Lawyers	UC-12
FR-16	Bulk review	UC-07
FR-17	Redline	UC-08
FR-18	Templates	UC-13
FR-19	Template DOCX	UC-13
FR-20	Sharing	UC-15
FR-21	Translation	UC-14
FR-22	Voice	UC-17

8. System Design (Diagrams)

This chapter presents the design of Nyaya Mitra. It begins with the layered architecture and then uses a sequence of standard diagrams to describe the system from different viewpoints: use case diagrams show who uses the system and for what, data flow diagrams show how data moves between users, processes and stores, UML diagrams show the structure and behaviour of the software, and the entity-relationship diagram shows the persistent data model. The chapter closes with the route, API, collection, implementation and interface specifications that tie the diagrams to the source code.

8.1 System Architecture

Nyaya Mitra follows a layered client-server architecture. The presentation layer is a React single-page application that handles navigation, forms and display. The application layer is a FastAPI service that authenticates every request, validates the input with Pydantic models and orchestrates the business workflow. A service layer groups the capabilities that the workflows depend on: the Gemini 3 Flash language model, the pypdf and python-docx document libraries, concurrent task execution for bulk review and the CNR decoder. The persistence layer consists of MongoDB, accessed through the Motor asynchronous driver, together with the external e-Courts portal that is reached only through a link and a reachability check.

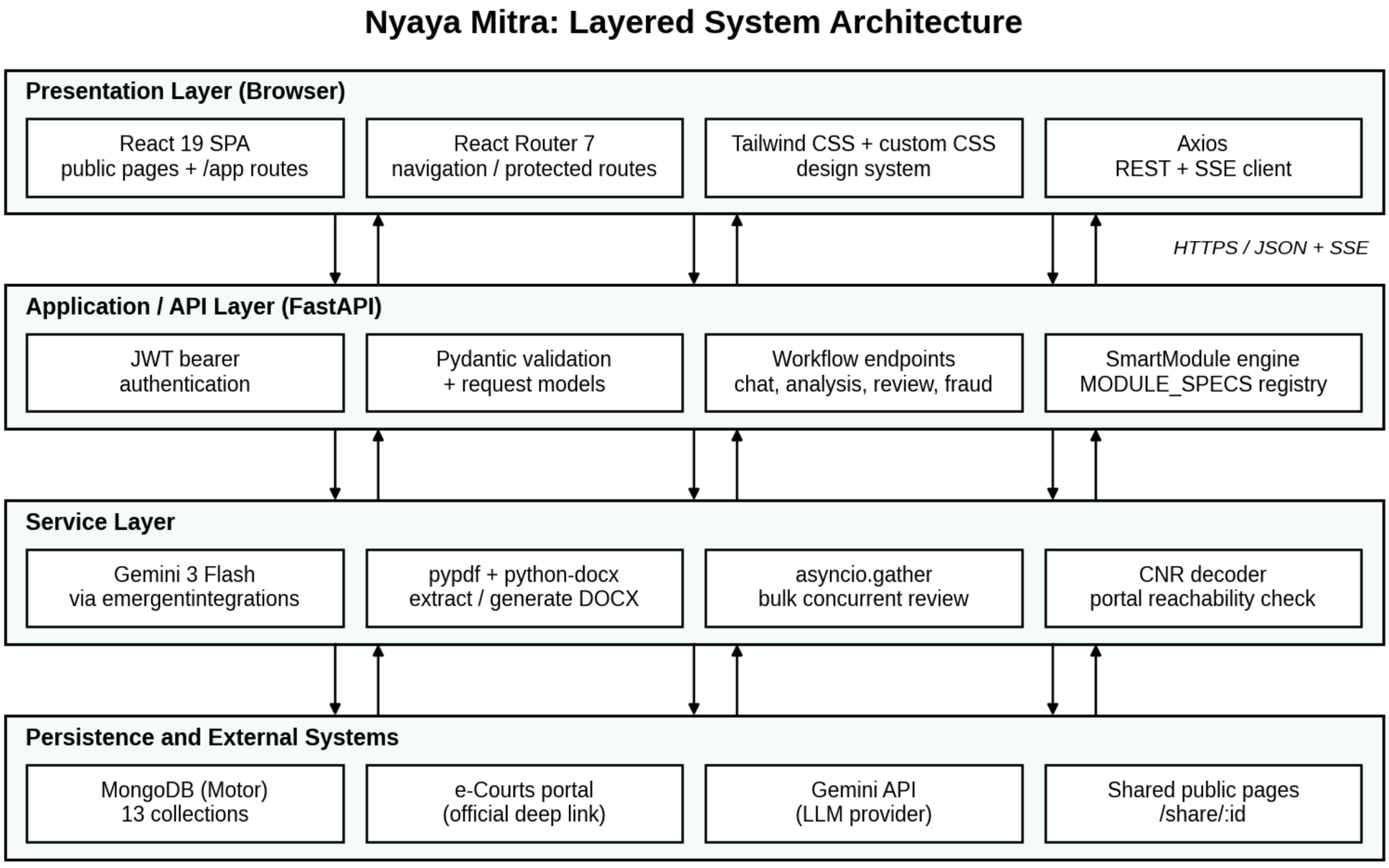


Figure 8.1. Layered system architecture of Nyaya Mitra

Figure 8.1 shows how a request travels through these layers. The browser sends JSON requests over HTTPS and, for the chatbot, opens a Server-Sent Events stream. The API layer checks the bearer token and the request body before any work is done. Only the backend communicates with the language model, which means that the provider key never reaches the browser and the backend can apply its own prompts, validation and error handling. Results are stored in MongoDB so that histories and cached reviews can be retrieved later.

Layer	Components	Responsibility
Presentation	React, React Router, Tailwind/custom CSS	UI and navigation.
Application/API	FastAPI, Pydantic	Validation, auth, workflow orchestration.
AI	Gemini 3 Flash / emergentintegrations	Generation, analysis, review, translation.
Persistence	MongoDB / Motor	Application records and histories.
Documents	pypdf, python-docx	Extraction and generated DOCX.
External	e-Courts portal	Official portal reachability/deep link.

Primary flow: User → React Frontend → FastAPI API → authentication/validation → business workflow → Gemini/MongoDB/document services → structured response → React UI. The browser does not directly call the LLM.

The table above summarises the responsibility of each layer. The separation has practical benefits: the frontend can be redesigned without touching business logic, the AI provider can be replaced by changing only the service layer, and the database can be moved from a local instance to a hosted cluster by changing a connection setting.

8.2 Use Case Diagram

A use case diagram shows the actors of a system and the functions that each actor can perform. In the diagrams below the rectangle represents the system boundary, the ellipses represent use cases and the lines connect an actor to the use cases that the actor can start. The identifiers UC-01 to UC-17 match the use case list in Section 7.1.

The first diagram shows the authenticated general user, who has access to all 17 use cases. Registration and login are included in the diagram because they are the means by which a visitor becomes an authenticated user.

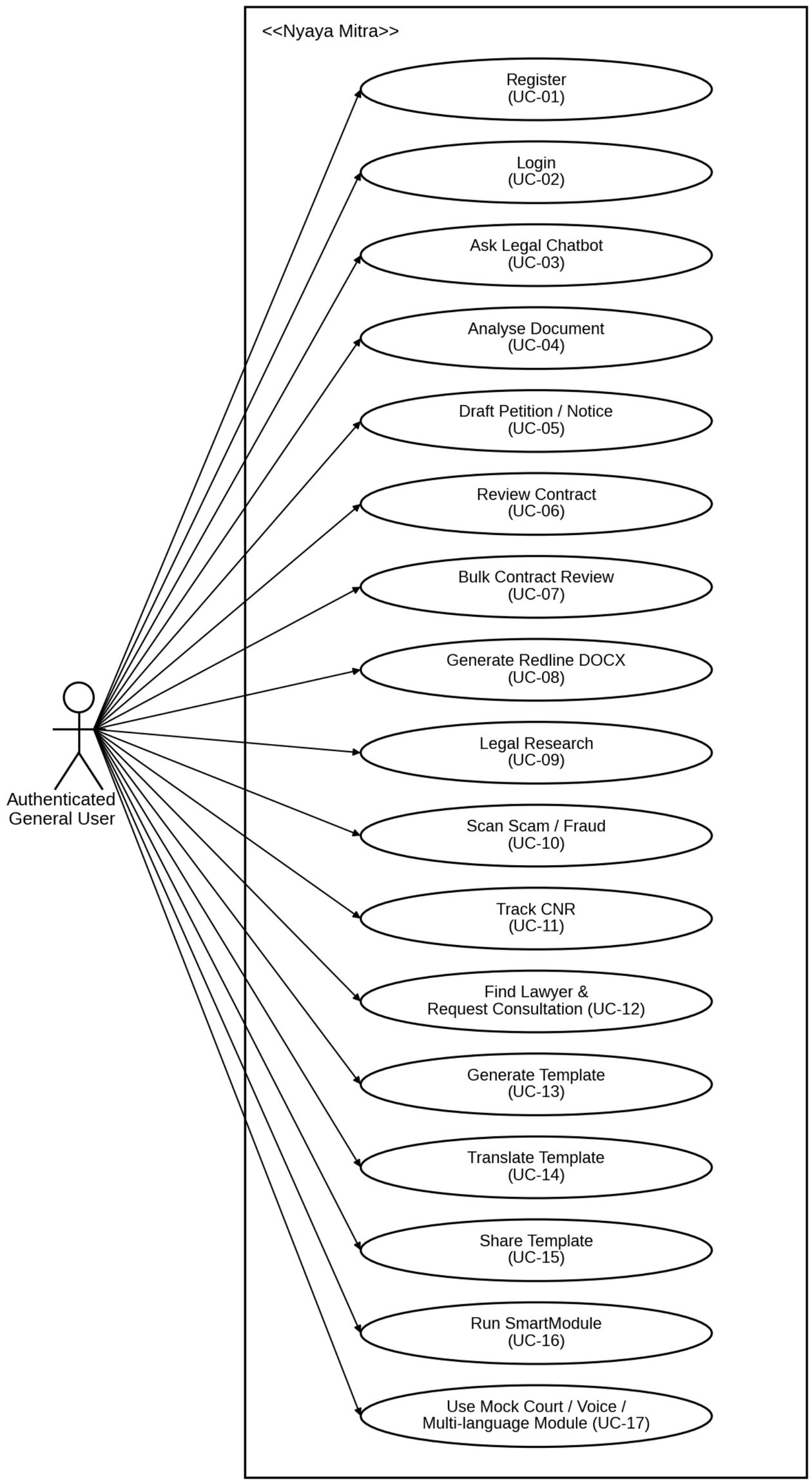


Figure 8.2. Use case diagram for the authenticated general user

The second diagram shows the use cases that are most relevant to a lawyer or legal professional. These are research, single and bulk contract review, drafting, case tracking through the CNR tracker, the client-management module and the lawyer marketplace. Technically the professional uses the same account type as every other user, so the diagram describes the workflows that are most relevant to that group rather than a separately enforced role.

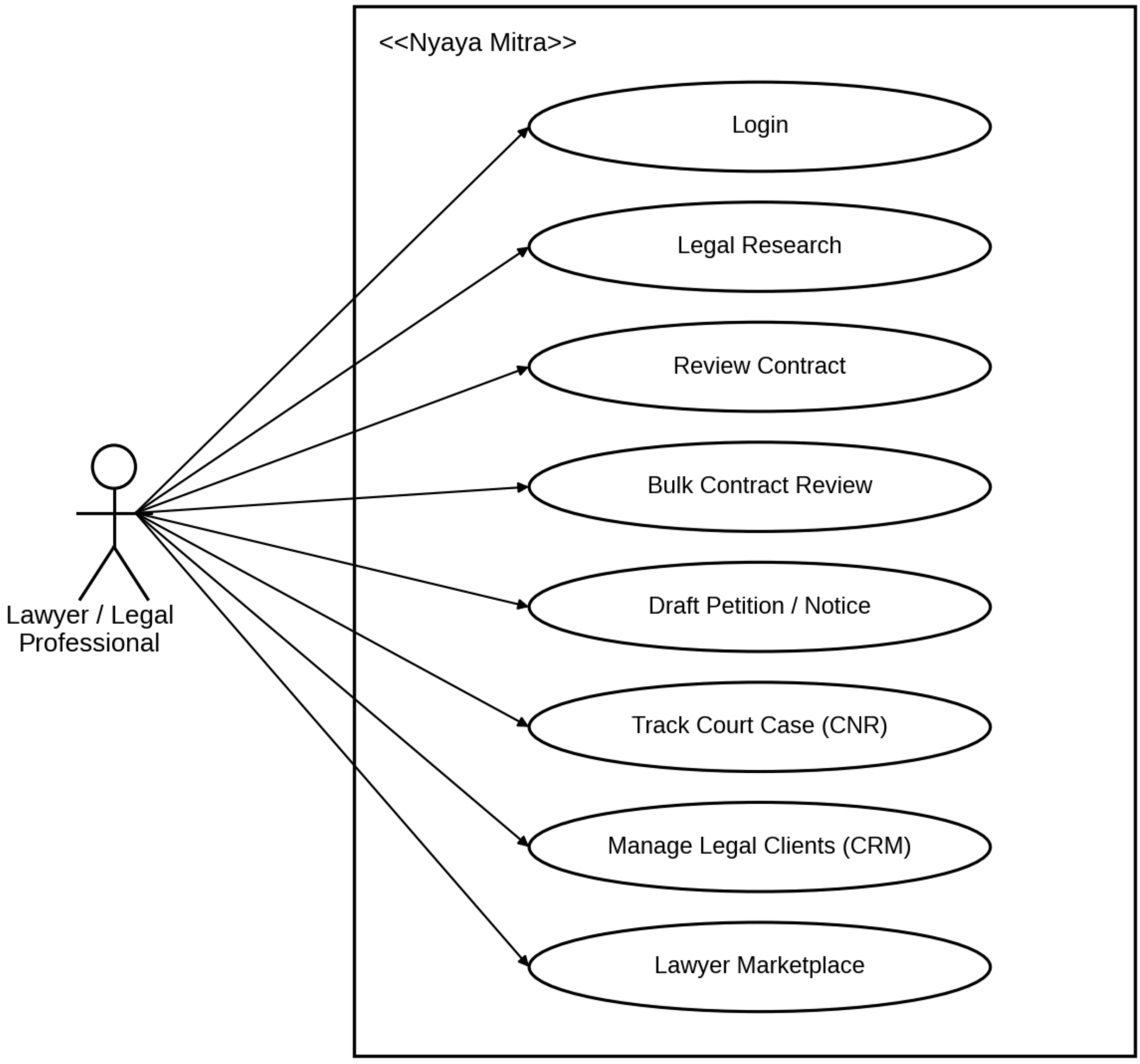


Figure 8.3. Use case diagram for the lawyer or legal professional

The third diagram shows the law student. The student is supported through the chatbot, the research engine, the Mock Court module and the multilingual support module, which together provide a learning-oriented

environment in which arguments can be practised and legal topics can be explained in more than one language. An administrator use case diagram is not provided because the current system stores no administrator role.

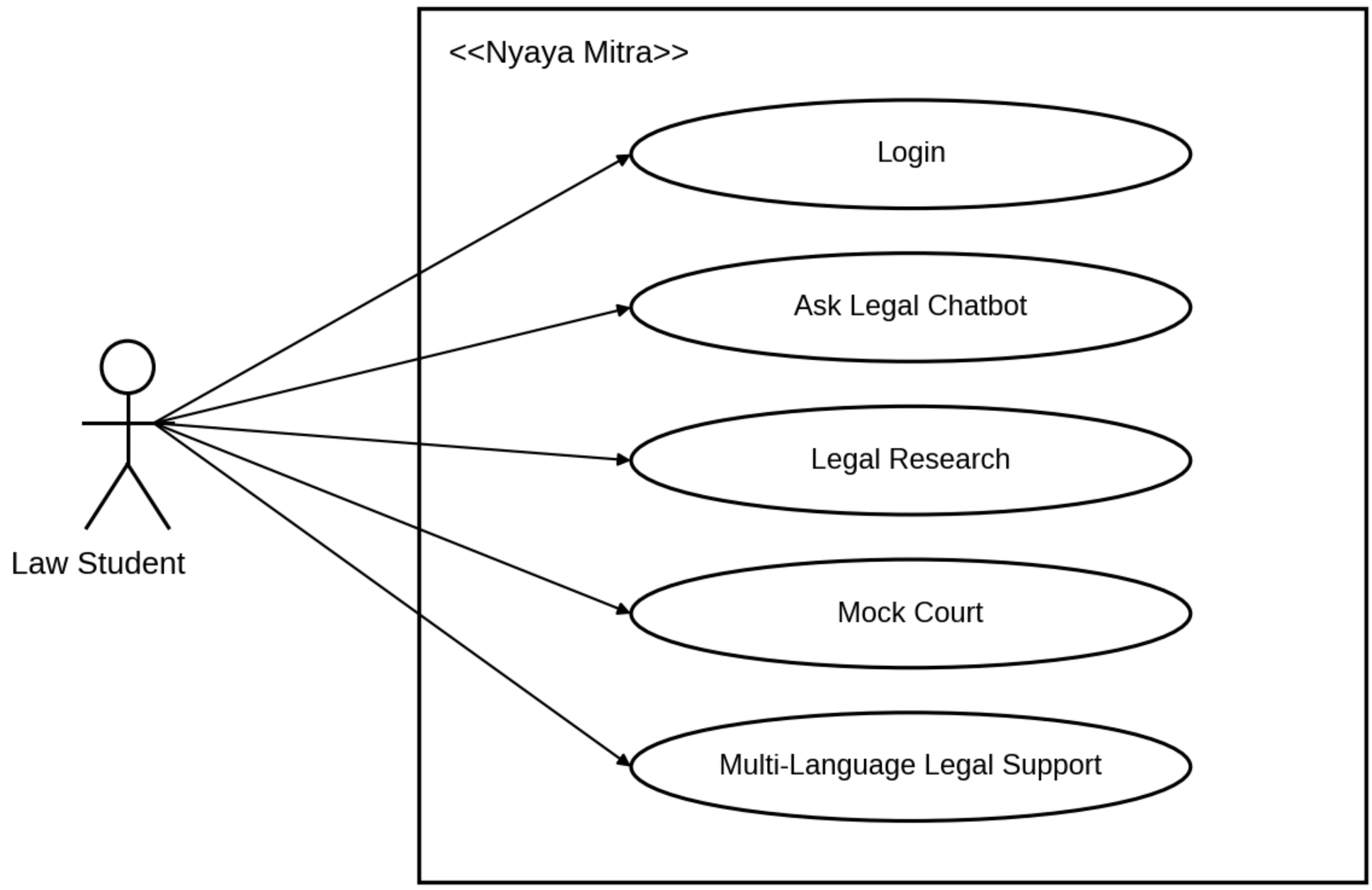


Figure 8.4. Use case diagram for the law student

8.3 Data Flow Diagrams

Data flow diagrams describe the movement of data through the system. The notation used in this document follows the standard Yourdon-style convention: a rectangle is an external entity, a circle with a number and a name is a process, two parallel horizontal lines represent a data store, and an arrow is a data flow carrying a label. The context diagram shows the whole system as one process, the level 1 diagrams divide it into its main processes, and the level 2 diagrams decompose selected processes in more detail.

8.3.1 Context Level (Level 0)

The context diagram shows Nyaya Mitra as a single process, 0.0, and identifies three external entities. The user sends credentials, legal queries, documents, contracts, scam content, CNR numbers and template requests, and receives tokens, streamed answers, analyses, drafts, risk results, DOCX files, lawyer lists and the official e-Courts link. The Gemini 3 Flash service receives prompts that contain user content and returns

structured responses. The e-Courts portal receives a reachability check for a CNR and returns the portal status.

Context Level DFD: 0 level

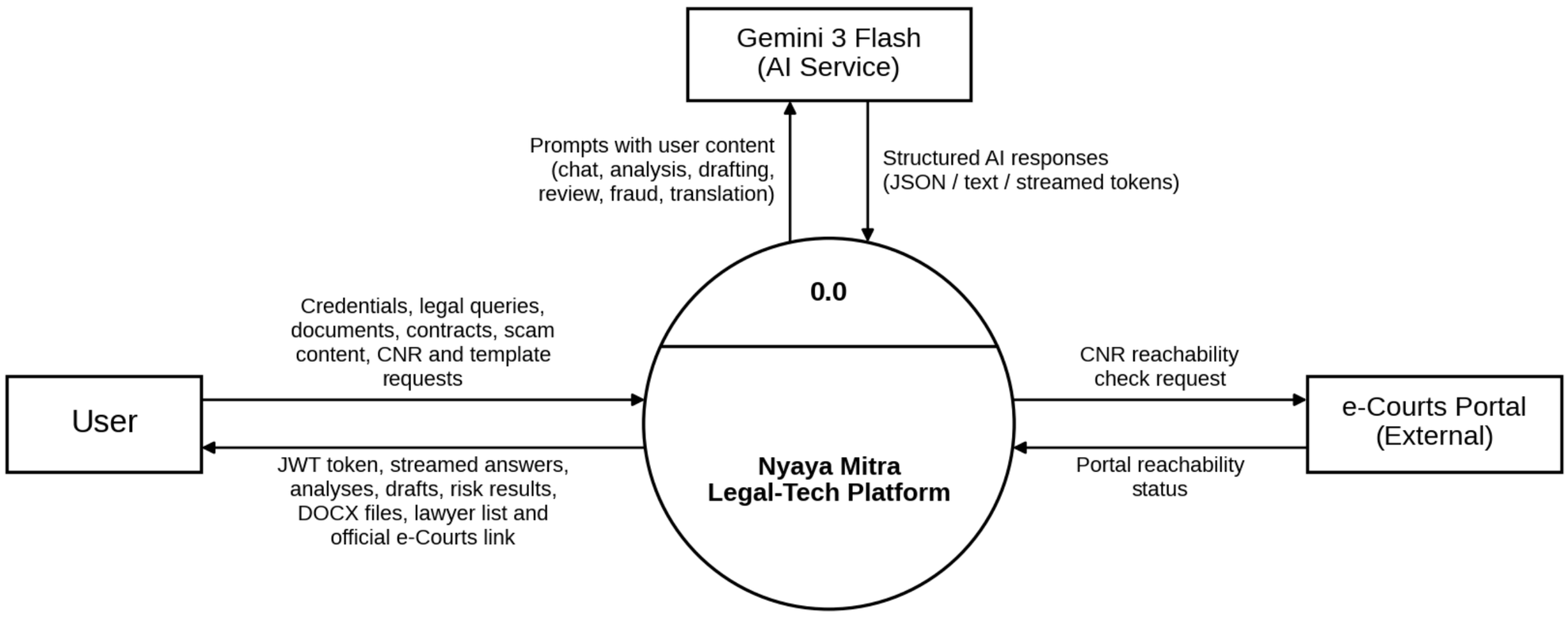


Figure 8.5. Context level DFD (level 0)

8.3.2 Level 1 Data Flow Diagrams

The level 1 diagrams divide the system into twelve processes. The first diagram covers the core workflows: registration and login (1.0), the AI legal chatbot (2.0), the document analyzer (3.0), contract review and redline (4.0), petition and notice drafting (5.0) and legal research (6.0). Each process exchanges a request and a response with the user and reads or writes one data store, namely users, chat_messages, analyses or drafts.

User Side DFD :- 1st Level (Core AI Workflows)

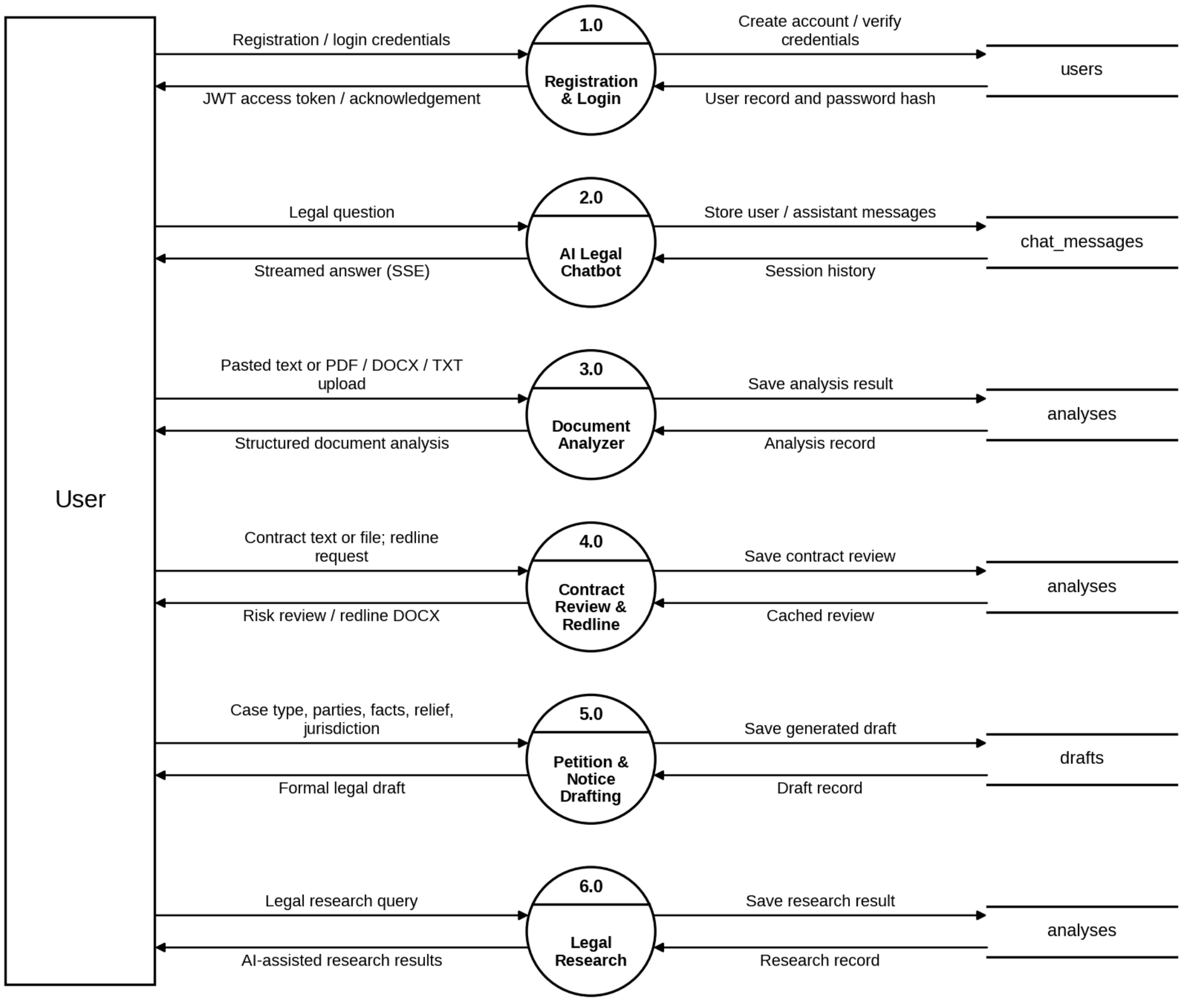


Figure 8.6. Level 1 DFD, user side: core AI workflows

The second level 1 diagram covers the advanced workflows: scam and fraud detection (7.0), the SmartModule engine (8.0), the CNR tracker (9.0), the lawyer directory (10.0), bulk contract review (11.0) and contract templates (12.0). Three of these processes use two data stores: fraud detection writes scans and community alerts, the lawyer directory reads lawyer records and writes consultation requests, and the template process stores generations and public shares.

User Side DFD :- 1st Level (Advanced AI and Fraud Workflows)

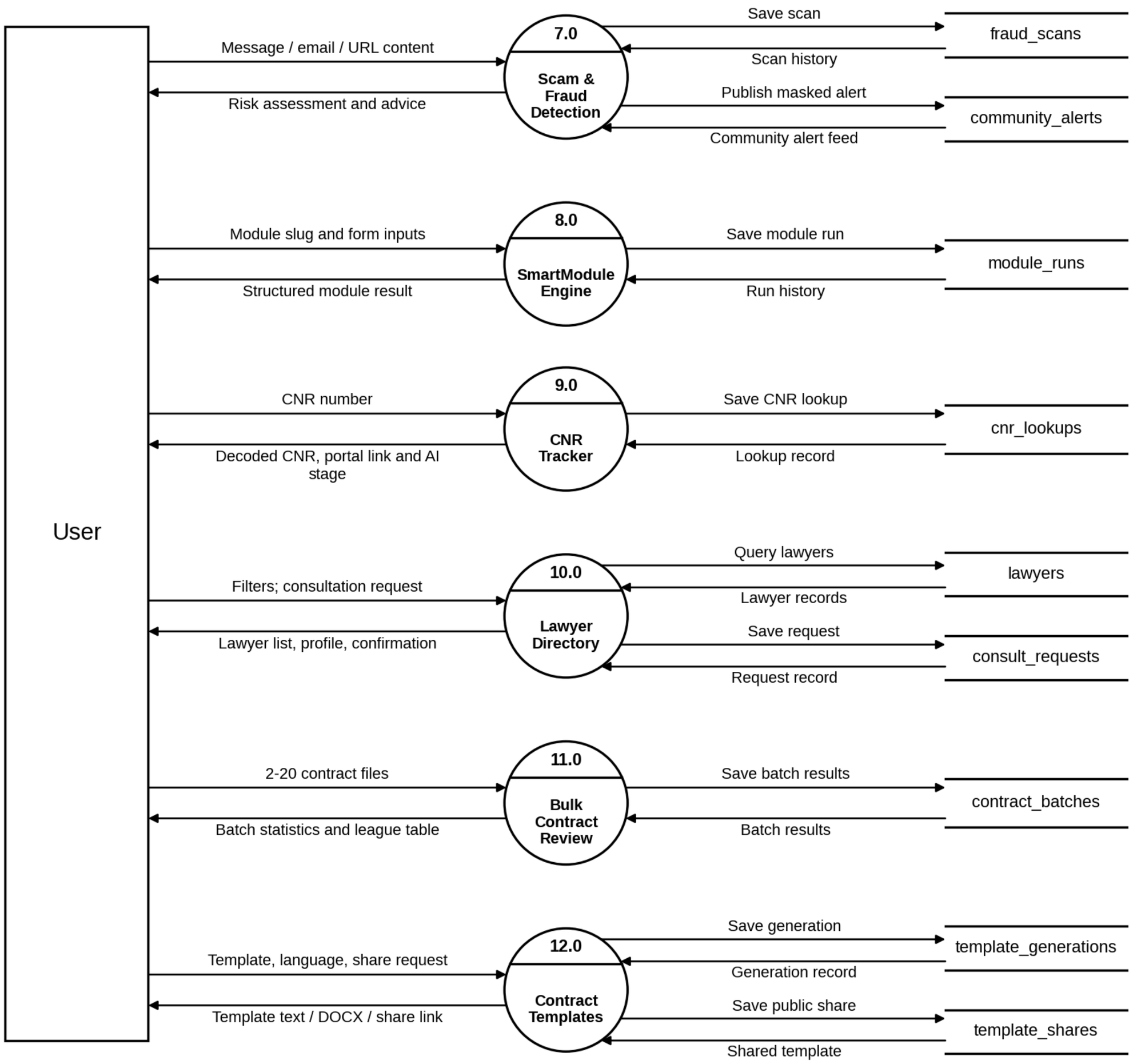


Figure 8.7. Level 1 DFD, user side: advanced AI and fraud workflows

8.3.3 Level 2 Data Flow Diagrams

Level 2 diagrams decompose three of the most important processes. The contract review and redline process (4.0) is broken into four sub-processes: submitting the contract and extracting its text (4.1), the AI risk review that calls Gemini (4.2), storing and displaying the review (4.3) and generating the redline DOCX

from the cached review (4.4). The decomposition makes visible that the extracted text flows from one sub-process to the next and that the stored analysis is reused to build the redline.

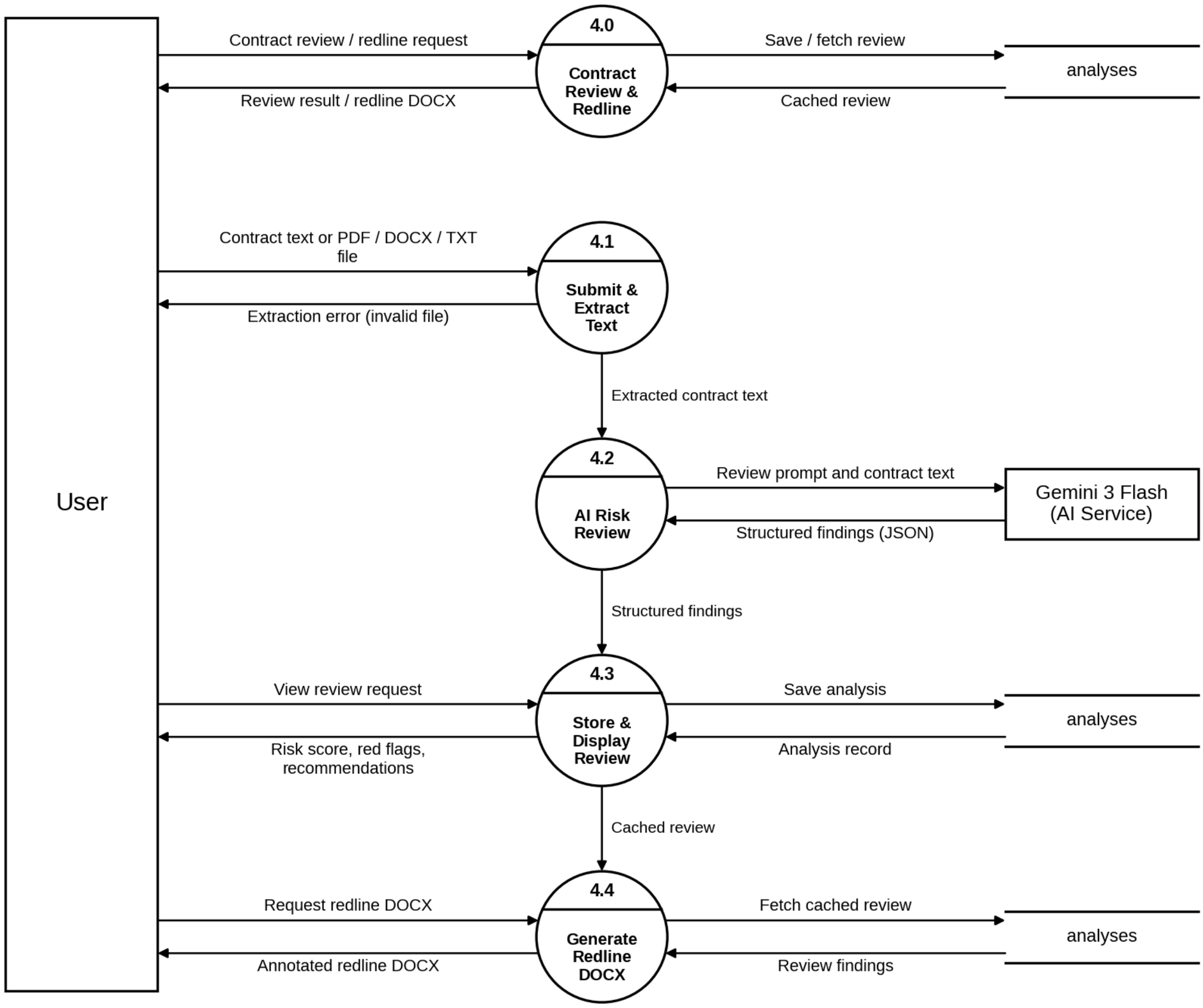


Figure 8.8. Level 2 DFD: contract review and redline

The scam and fraud detection process (7.0) is decomposed into content submission (7.1), AI risk assessment (7.2), storing the scan and showing the result (7.3) and publishing a masked community alert (7.4). Sub-process 7.4 is entered only when the result is suspicious or dangerous, and it also serves the user who views the community alert feed.

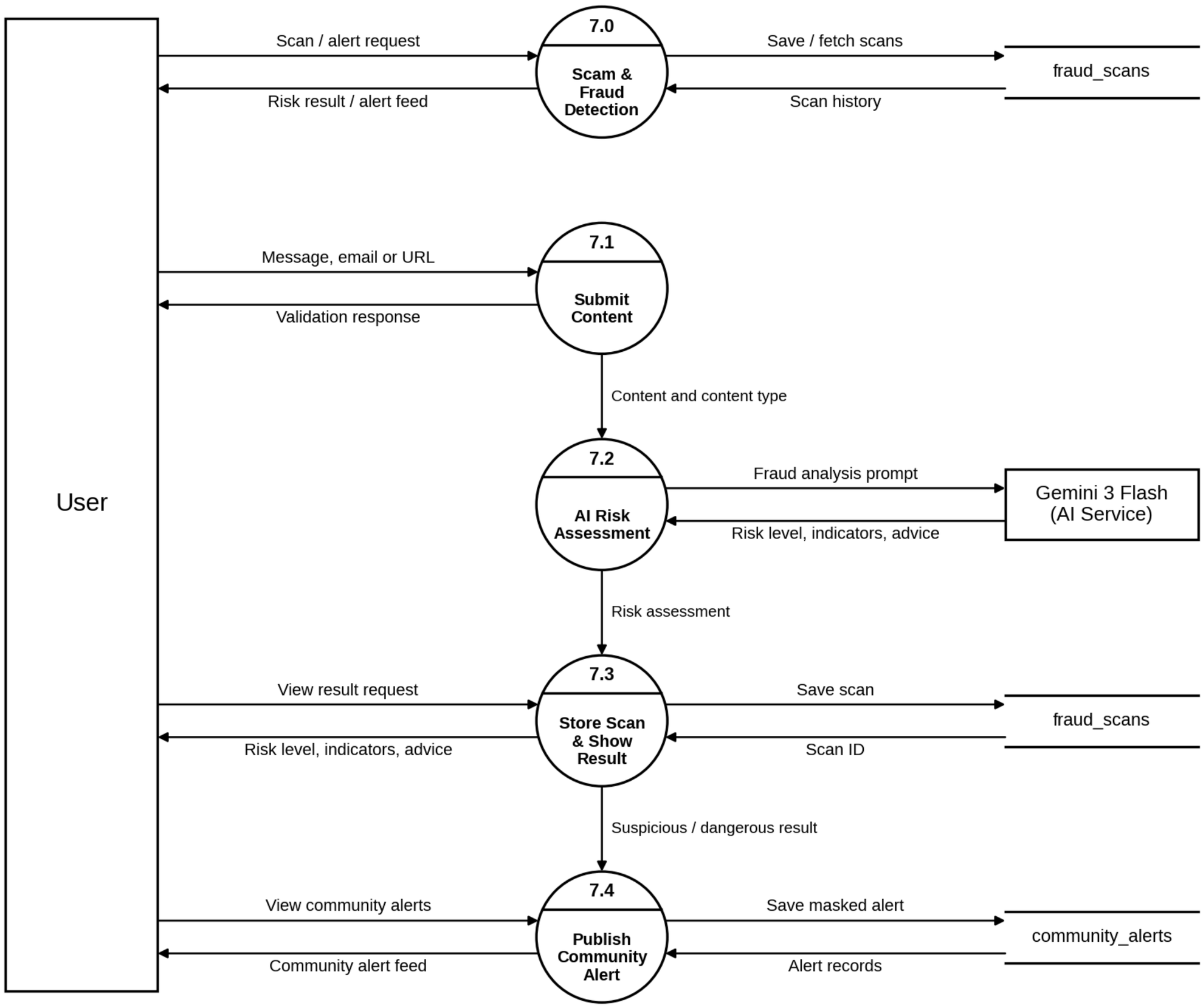


Figure 8.9. Level 2 DFD: scam and fraud detection

The contract templates process (12.0) is decomposed into template generation by the language model (12.1), DOCX download (12.2), translation into a regional language (12.3), creation of a public share (12.4) and viewing of a shared template (12.5). The last sub-process can be reached through a public link and reads only the stored share, which contains the title and body that the owner chose to publish.

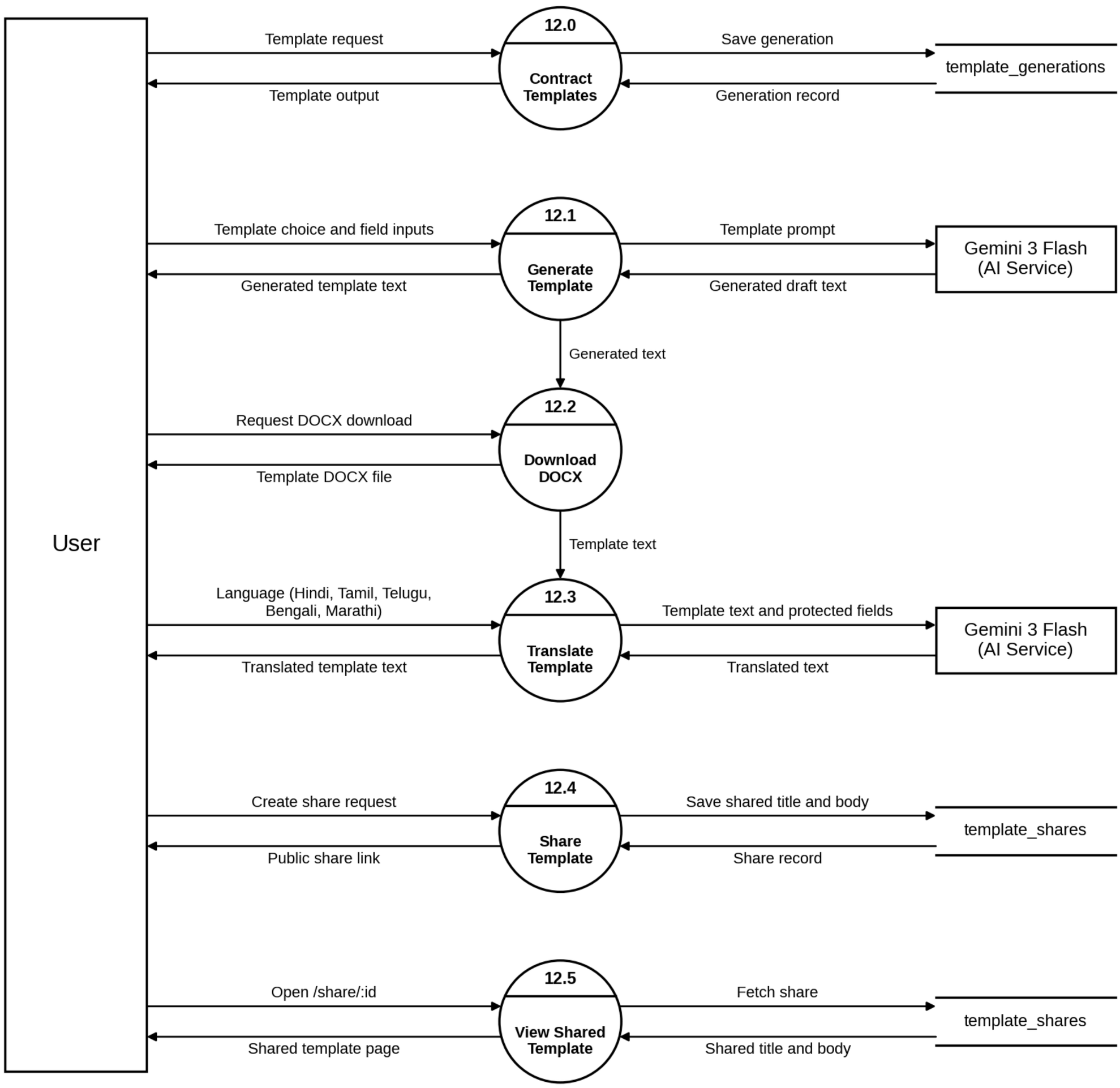


Figure 8.10. Level 2 DFD: contract templates

8.4 UML Diagrams

The Unified Modeling Language is used to describe the structure of the software with a class diagram, the interaction between components with sequence diagrams and the flow of control with activity diagrams.

Together they complement the data flow diagrams by showing how the same system is organised and how it behaves over time.

8.4.1 Class Diagram

The class diagram represents the principal persistent entities of the system and the operations associated with them. The User class is the centre of the model: it owns chat messages, analyses, drafts, module runs, CNR lookups, fraud scans, contract batches, template generations and consultation requests through one-to-many associations. Two further associations connect a fraud scan to the community alert that it may raise, and a template generation to the public shares created from it. A consultation request links a user with a lawyer from the seeded directory.

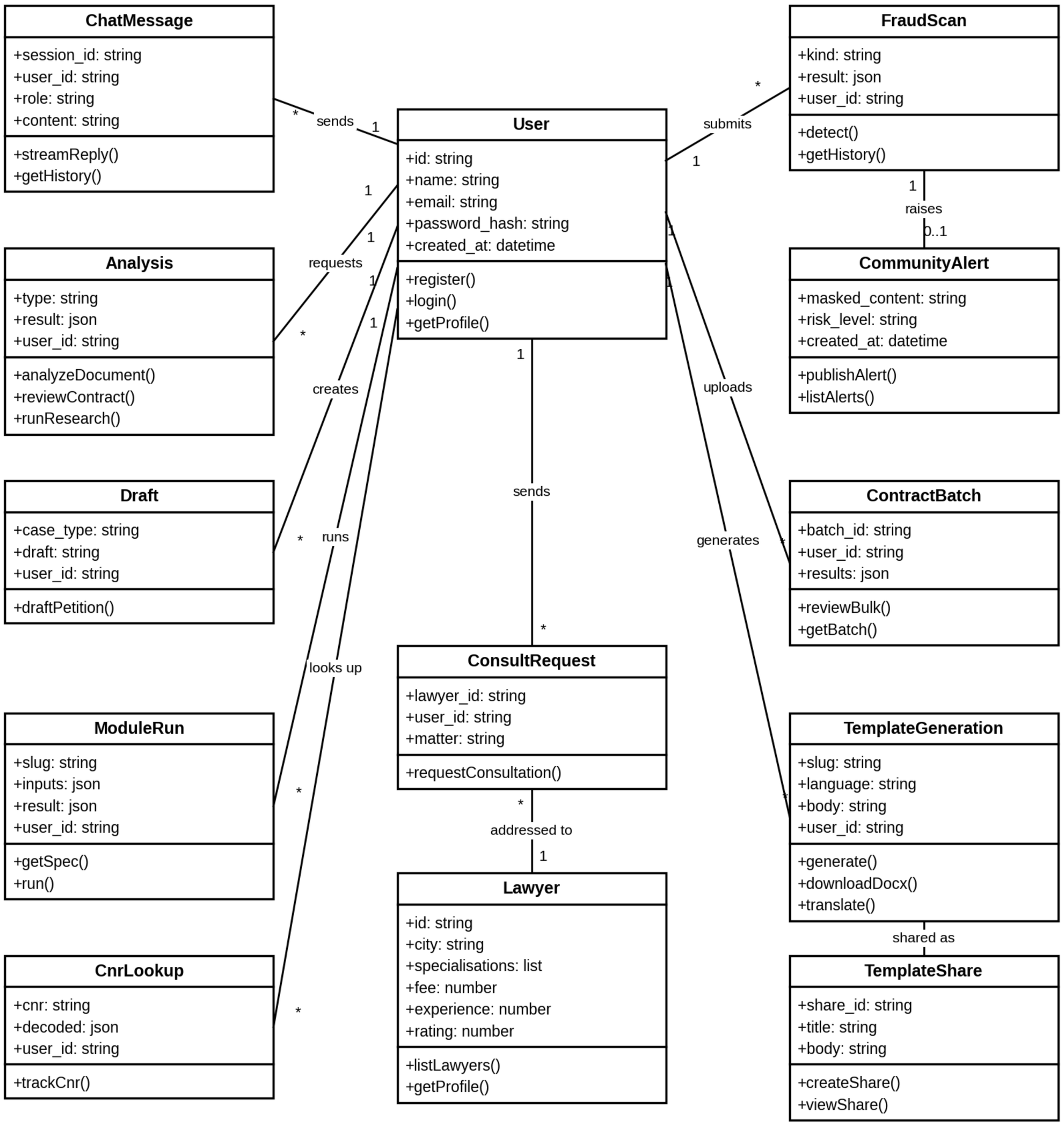


Figure 8.11. Class diagram of Nyaya Mitra

The attributes shown in each class follow the core data dictionary in Section 8.6, and the operations correspond to the endpoint groups in Section 8.7. Where the dictionary does not list every field, for example

the community alert and CNR lookup classes, the diagram shows the fields that the corresponding workflow evidently requires.

8.4.2 Sequence Diagrams

Sequence diagrams show the order of messages exchanged between the user, the frontend, the backend and the supporting services for one scenario. Time flows from top to bottom, solid arrows are requests and dashed arrows are responses. Four scenarios are shown: authentication, the streaming chatbot, contract review with redline generation, and fraud detection.

In the authentication sequence the user enters credentials in the React frontend, which calls the register or login endpoint. The backend validates the input, hashes or verifies the password, reads or writes the user document in MongoDB and issues a signed JWT. The frontend then keeps the token and redirects the user to the dashboard.

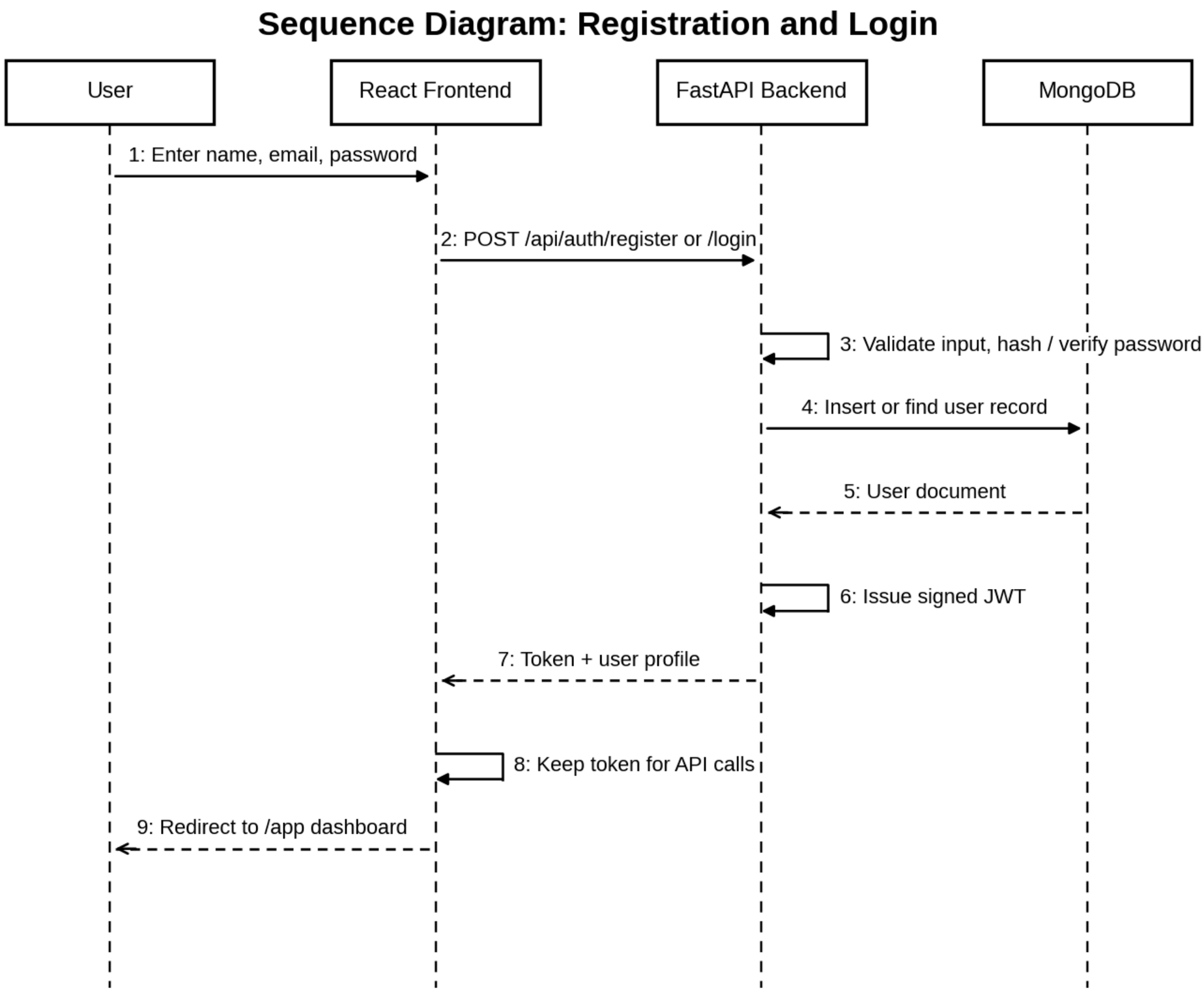


Figure 8.12. Sequence diagram: registration and login

The chatbot sequence differs from the others because the response is streamed. After verifying the token and creating or reusing a session, the backend saves the user message, sends the prompt together with the conversation history to Gemini and relays the generated tokens to the browser as Server-Sent Events. The assistant message is saved once the stream is complete, so that the full conversation is available in the history.

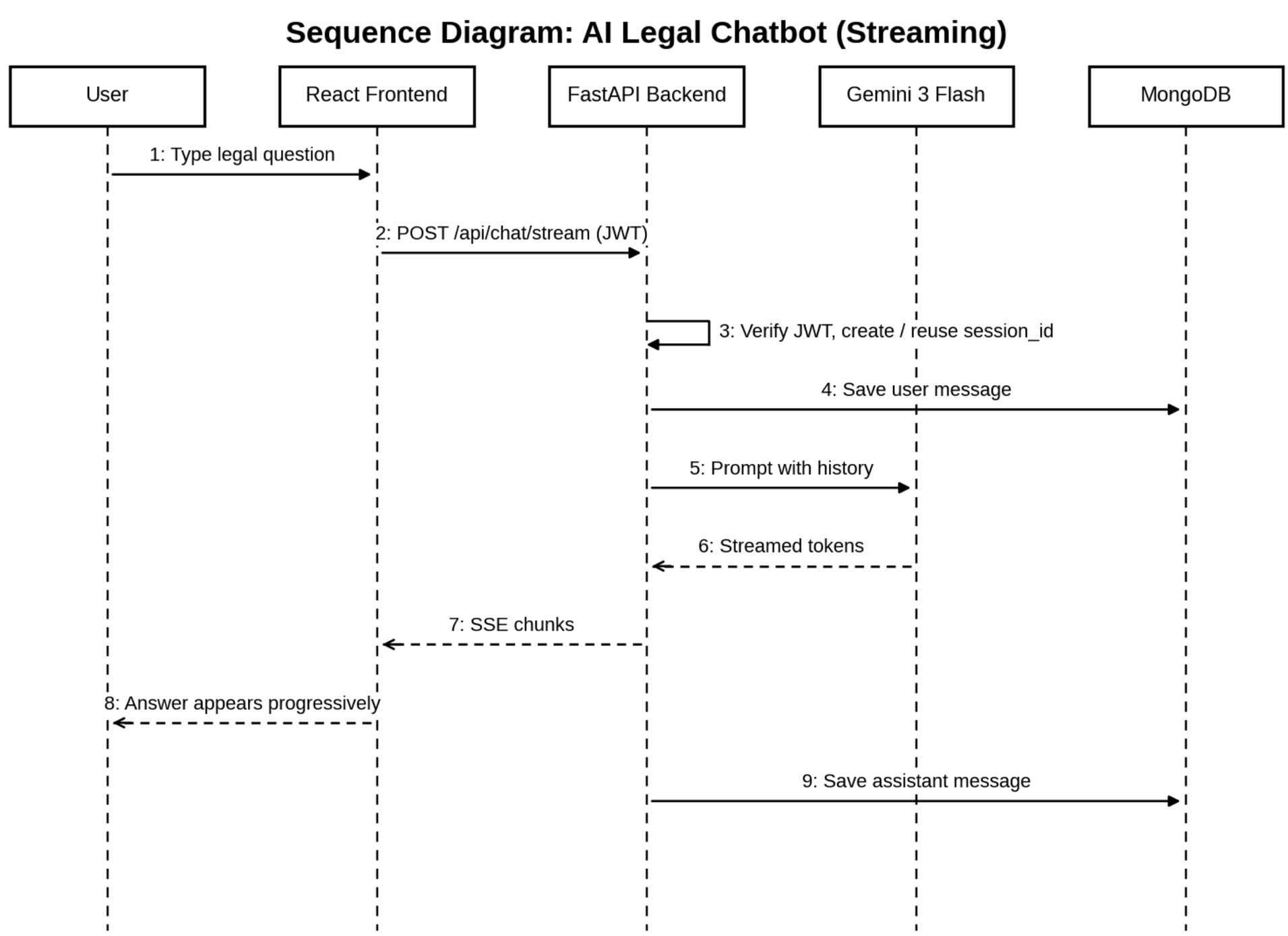


Figure 8.13. Sequence diagram: AI legal chatbot with streaming

The contract sequence has two phases. In the review phase the backend extracts text with pypdf or python-docx, asks Gemini for structured findings, stores the analysis and returns the risk score, red flags and recommendations. In the redline phase the backend reuses the stored review, builds the annotated DOCX with python-docx and returns it as a download.

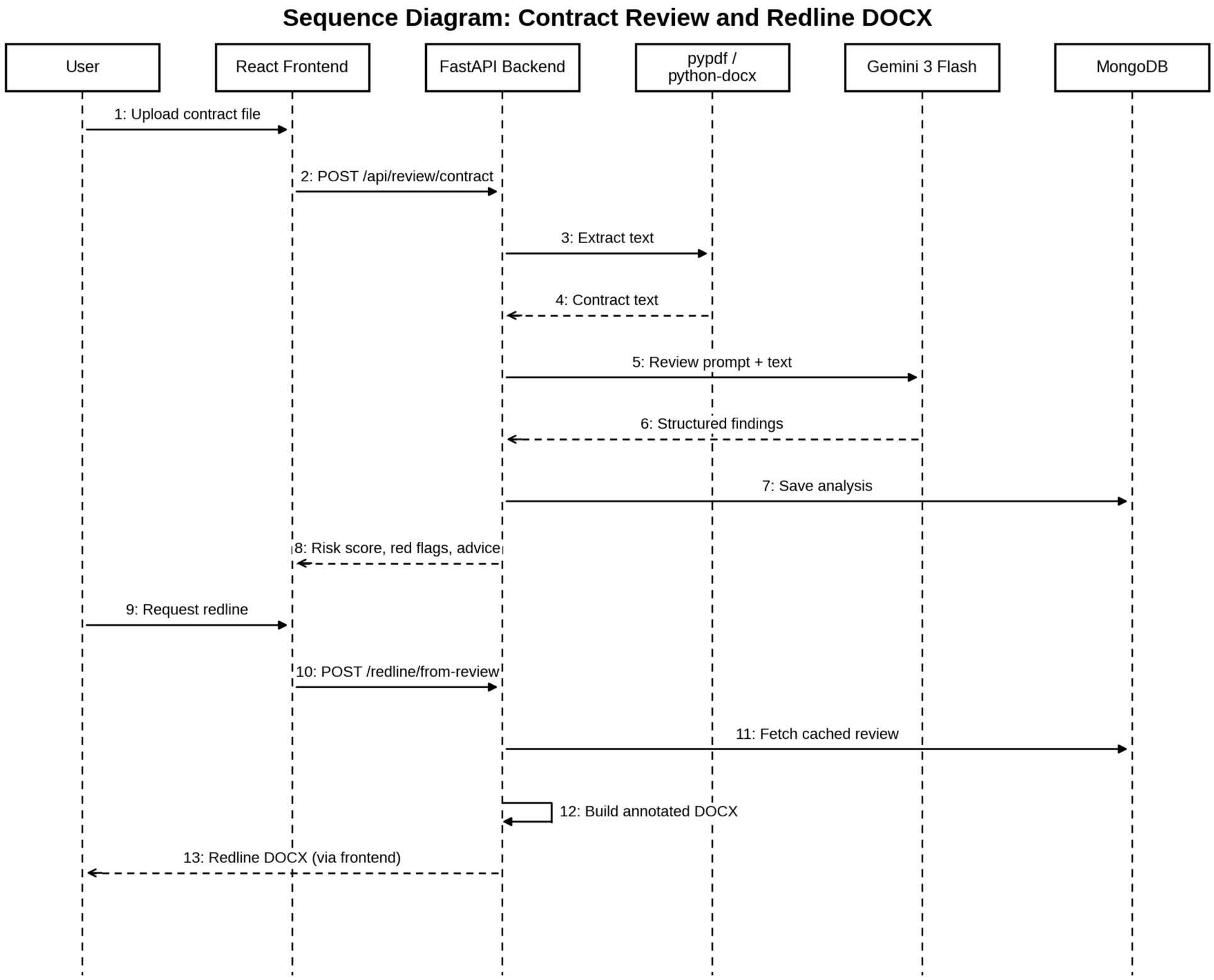


Figure 8.14. Sequence diagram: contract review and redline DOCX

In the fraud sequence the backend sends a risk-assessment prompt to Gemini, stores the scan and, only when the risk is suspicious or dangerous, stores a masked community alert before returning the result. The alternative fragment in the diagram expresses this condition.

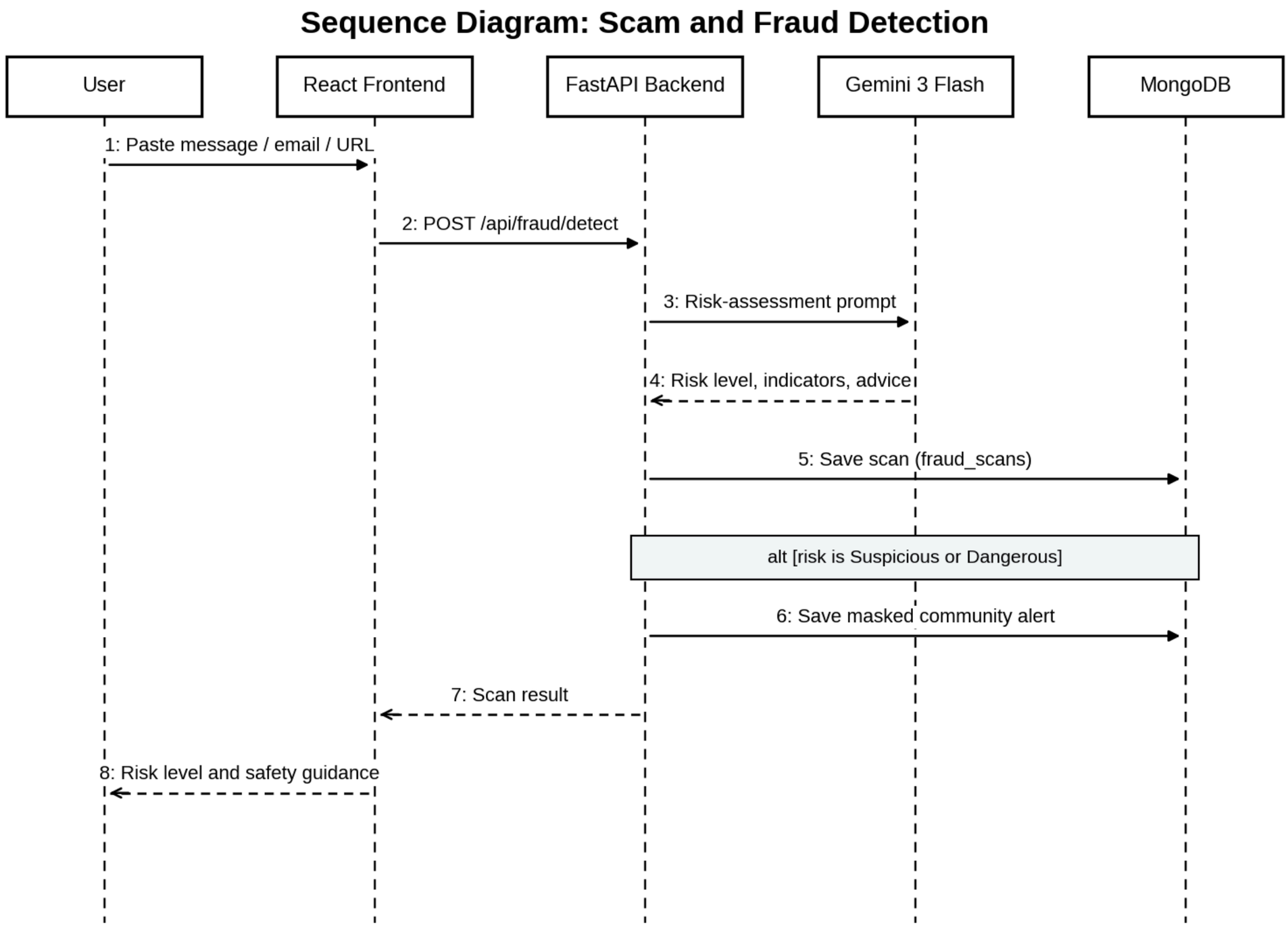


Figure 8.15. Sequence diagram: scam and fraud detection

8.4.3 Activity Diagrams

Activity diagrams describe the flow of control within a workflow, including decisions and parallel activities. The first diagram shows the overall behaviour of a user session. After login and a successful authentication check, the user may start any of the module workflows, shown as parallel activities between the fork and the join bars, and finally logs out. An invalid authentication returns the user to the login step.

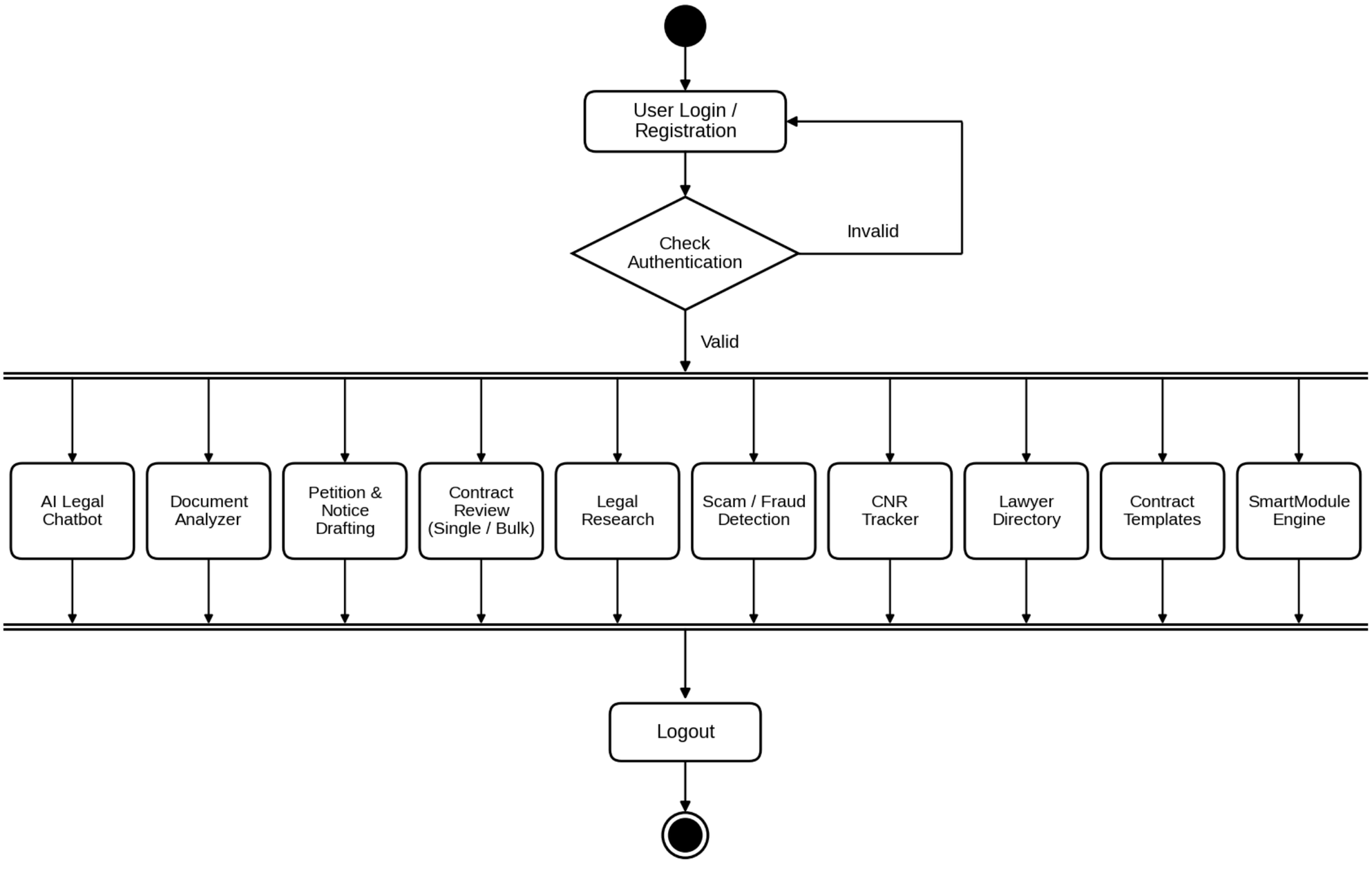


Figure 8.16. Activity diagram: user session

The second activity diagram shows the contract review and redline workflow. The user submits pasted text or a file; uploaded files pass through text extraction, and the text is validated before it is sent to the language model. A rejected input returns the user to the submission step. After the review is stored and displayed, the user may optionally generate and download the annotated redline.

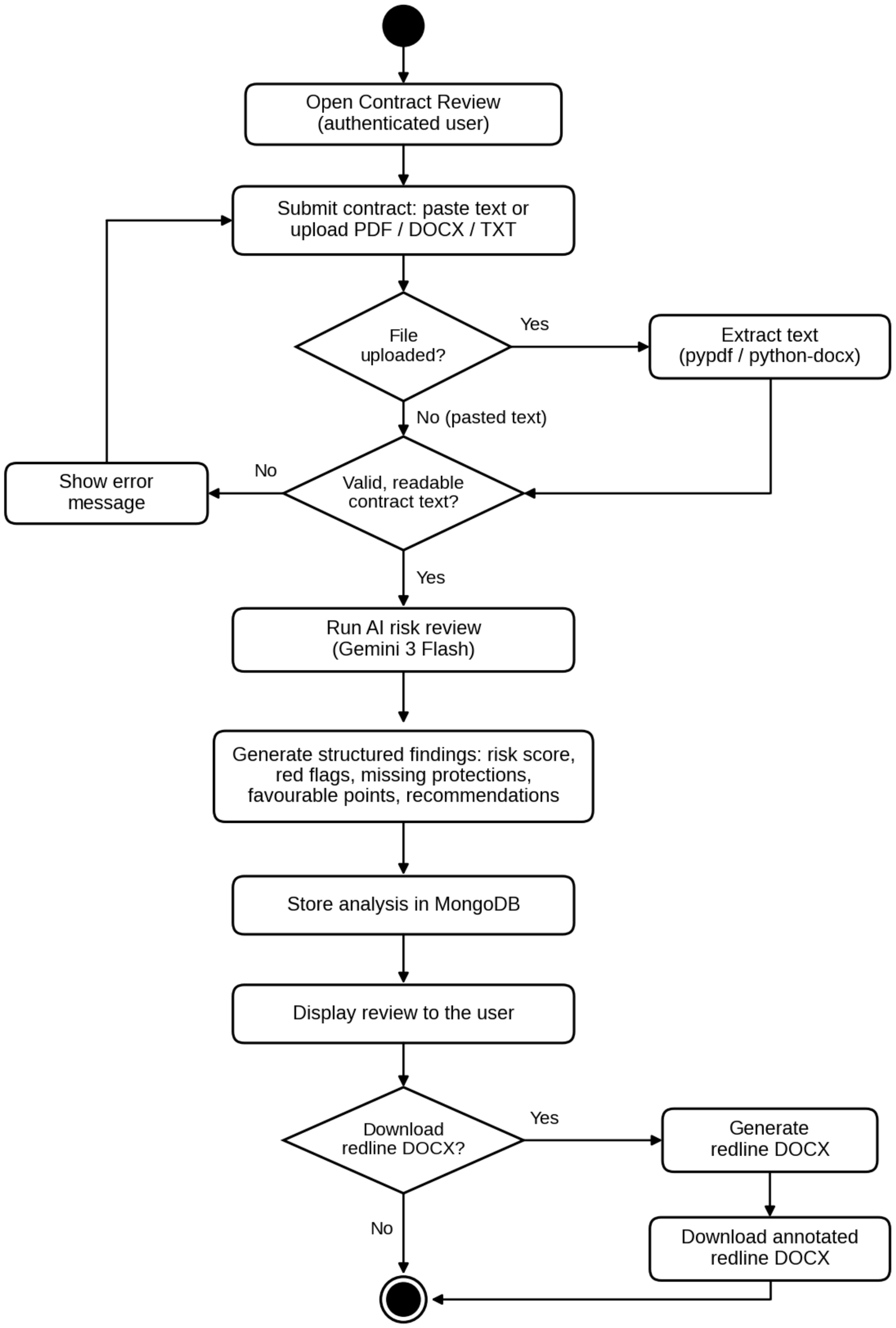


Figure 8.17. Activity diagram: contract review and redline

The third activity diagram shows the fraud detection workflow. The input is validated, assessed by the model and stored. The risk level, scam indicators and advice are displayed, and a decision node determines whether a masked community alert is created when the result is suspicious or dangerous.

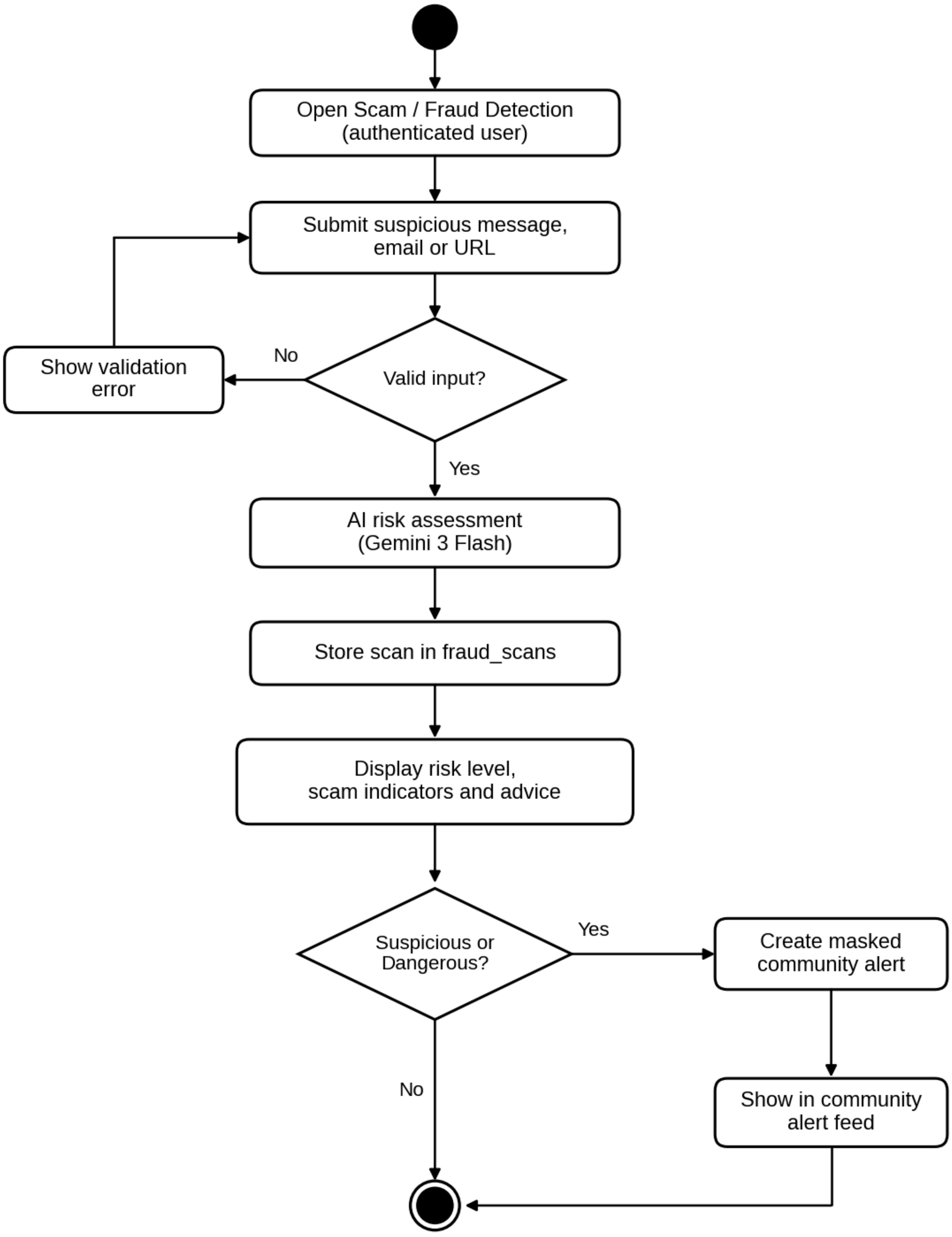


Figure 8.18. Activity diagram: scam and fraud detection

8.5 E-R Diagram

The entity-relationship diagram models the MongoDB data of Nyaya Mitra using Chen notation: rectangles are entities, ellipses are attributes, diamonds are relationships, an underlined attribute is the key, and the labels 1 and N show cardinality. The model has 13 entities. To keep the diagram readable it is presented in two views.

This E-R diagram represents the data model of the system. The central entity is users, which stores account identity and the password hash. Every user-scoped record in the system is linked to a user through a one-to-many relationship.

The first view shows the core workflow entities: chat_messages (conversation history by session), analyses (document, contract and research analyses), drafts (petition and notice drafts), module_runs (generic SmartModule executions) and cnr_lookups (CNR tracking records). The second view shows fraud_scans with the community_alerts they may raise, contract_batches for bulk review, consult_requests addressed to lawyers from the seeded lawyer directory, and template_generations with their public template_shares.

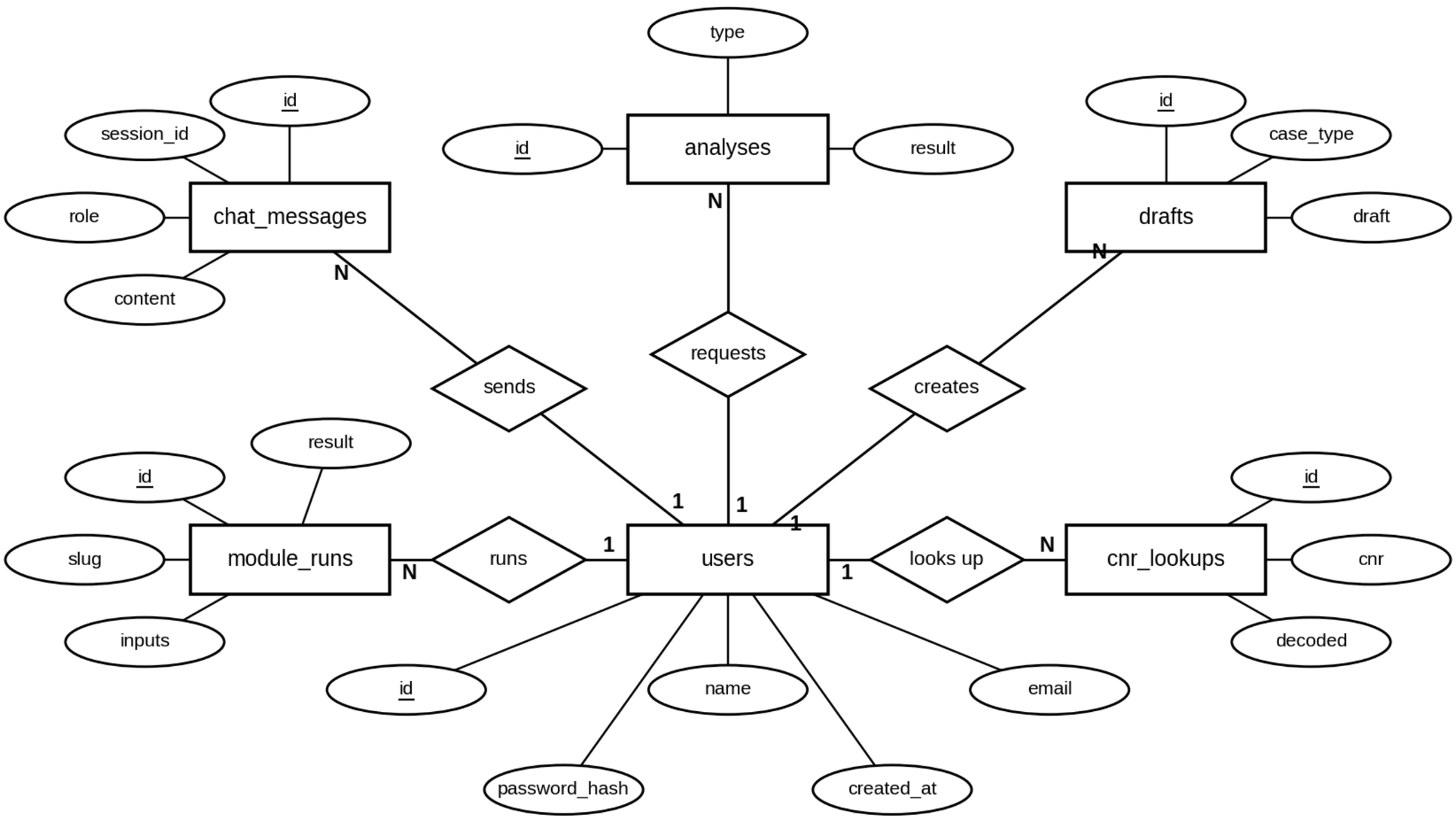


Figure 8.19. E-R diagram: core user workflows

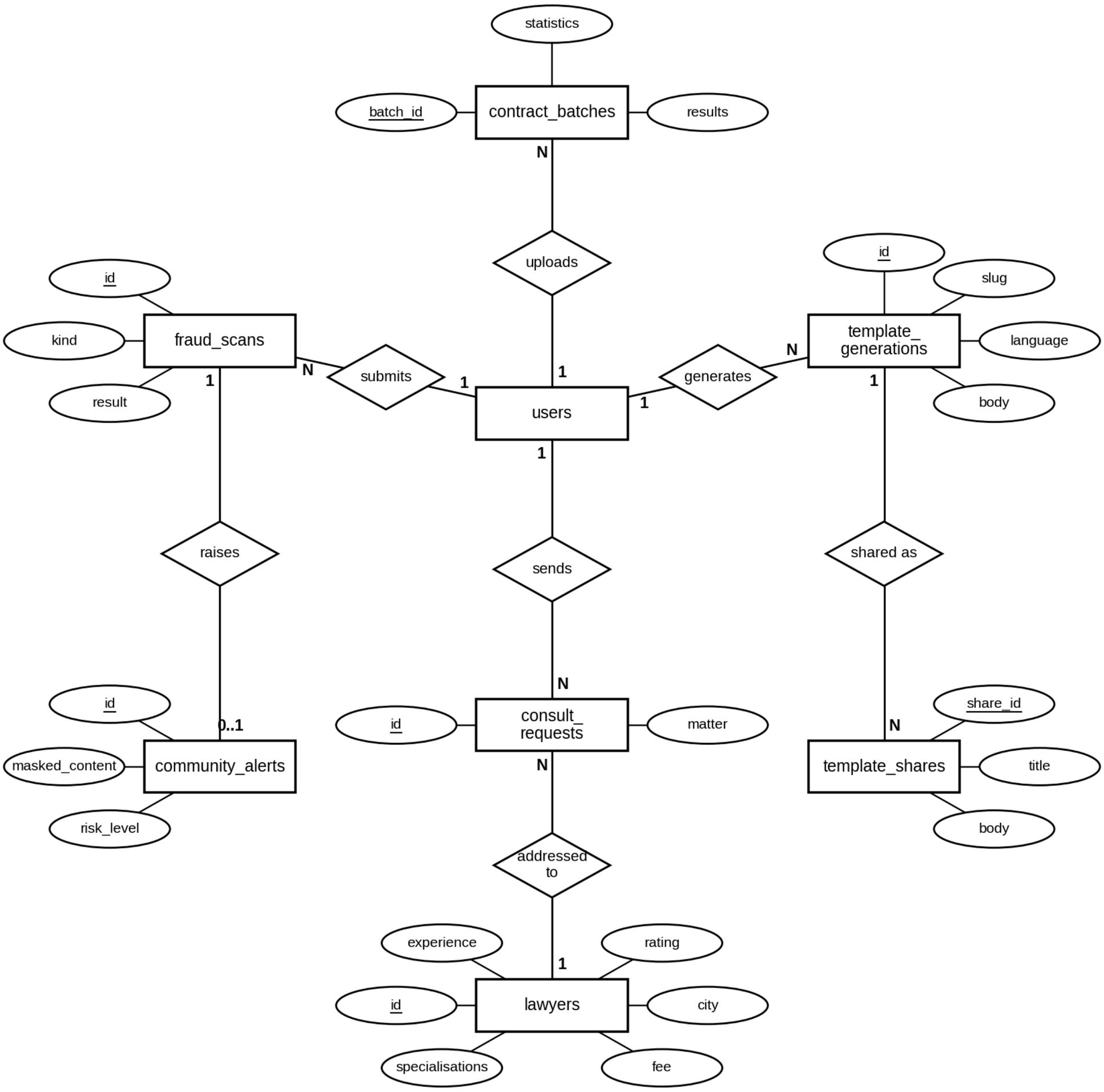


Figure 8.20. E-R diagram: fraud, bulk review, lawyer and template entities (attributes of users are shown in Figure 8.19)

Because MongoDB is a document database, relationships are implemented by storing the user identifier or another reference inside the child document rather than by foreign-key constraints. The ER diagram expresses these references as relationships so that the model can be understood independently of the storage technology.

8.6 Database Collections and Data Dictionary

The persistent state of the application is held in 13 MongoDB collections. The table below gives the purpose of each collection, and the data dictionary that follows lists the principal fields of the main entities.

Collection	Purpose
users	Account identity and password hash.
chat_messages	Chat messages and sessions.
analyses	Document/contract/research analyses.
drafts	Petition/notice drafts.
fraud_scans	Fraud scan history.
community_alerts	Public fraud alerts.
module_runs	Generic module executions.
contract_batches	Bulk review results.
cnr_lookups	CNR lookup records.
lawyers	Seeded lawyer directory.
consult_requests	Consultation requests.
template_generations	Generated templates.
template_shares	Public shared templates.

Entity	Field	Description
USER	id/name/email/password_hash/created_at	Account identity and secure authentication fields.
CHAT_MESSAGE	session_id/role/content	Conversation history.
ANALYSIS	type/result/user_id	AI analysis records.
DRAFT	case_type/draft/user_id	Generated legal drafts.
FRAUD_SCAN	kind/result/user_id	Fraud assessment.
MODULE_RUN	slug/inputs/result/user_id	Generic module execution.

LAWYER	id/city/specialisations/fee/experience/rating	Directory profile.
CONSULT_REQUEST	lawyer_id/user_id/matter	Consultation request.
TEMPLATE_SHARE	share_id/body/title	Public shared template content.

Documents in the user-owned collections include the identifier of the owner so that queries can be restricted to the authenticated user. The public collection template_shares is the exception: it holds only the title and body that a user has explicitly chosen to share, and it can be read without authentication through the share route.

8.7 Frontend Routes and API Design

The frontend routes define the pages of the application. Public routes are limited to the landing page, the authentication page and the public template share. All other routes are placed under /app and are protected by the authentication guard.

Route	Purpose
/	Landing
/auth	Login/registration
/share/:id	Public template share
/app	Dashboard
/app/chatbot	Legal chatbot
/app/document-analyzer	Document analyzer
/app/petition-draft	Petition drafting
/app/contract-review	Contract review
/app/research	Legal research
/app/cnr	CNR tracker
/app/marketplace	Lawyer directory
/app/bulk-contracts	Bulk review
/app/templates	Templates

/app/voice	Voice-style assistant
/app/fraud/:slug	Fraud subtype
/app/module/:slug	Generic SmartModule

The backend API is organised in groups that correspond to the workflows. Each group exposes a small number of endpoints, and all of them except the authentication endpoints and the public share view require the bearer token.

Group	Representative Endpoints
Auth	/api/auth/register, /login, /me
Chat	/api/chat/stream, /history/{session_id}, /sessions
Analysis	/api/analyze/document, /upload, /draft/petition, /review/contract
Research/Fraud	/api/research/query, /api/fraud/detect, /community, /history
Modules	/api/modules, /api/module/{slug}/spec, /api/module/run, /history/{slug}
Bulk	/api/review/contract/bulk, /review/batch/{id}, /review/batches
CNR	/api/cnr/track
Lawyers	/api/lawyers, /facets, /{id}, /consult
Redline	/api/review/contract/redline, /from-review, /redline-upload
Templates	/api/templates, /{slug}, /{slug}/generate, /{slug}/docx
Translation/Share	/api/languages, /translate, /docx-from-text, /share, /share/{id}

The API follows REST conventions for ordinary requests, uses multipart upload for files, returns generated documents as binary downloads and uses Server-Sent Events only for the chatbot stream. The module endpoints (/api/modules, /api/module/{slug}/spec and /api/module/run) allow a single frontend page to serve all SmartModules.

8.8 Implementation Specifications

This section records how the principal workflows are implemented in the backend. It complements the diagrams by stating the concrete behaviour that is verified in the test cases of Chapter 9.

8.8.1 Authentication

- Registration uses name, email and password.
- Backend currently enforces a minimum six-character password.
- Passwords are hashed with bcrypt/passlib.
- Login returns JWT.
- Protected APIs use HTTP Bearer authentication.
- Current registration does not store a formal user role.

Authentication is deliberately simple and stateless. The password is hashed before storage and only the hash is compared at login, and the signed token carries the identity needed to authorise each later request.

8.8.2 AI Chatbot

- POST /api/chat/stream provides SSE streaming.
- Session IDs are created or reused.
- User and assistant messages are stored.
- Recent sessions and session history can be retrieved.

Streaming keeps the interface responsive for long answers, and persistence of both sides of the conversation allows later review of a session.

8.8.3 Document Analyzer

- Pasted text and PDF/DOCX/TXT uploads are supported.
- PDF extraction uses pypdf.
- DOCX paragraphs and tables are extracted with python-docx.
- AI returns structured analysis that is persisted.

The extraction step converts every supported file into plain text so that the same analysis prompt can be usedfor pasted text and for uploads.

8.8.4 Petition and Notice Drafting

The drafting API accepts case type, parties, facts, relief sought and jurisdiction and generates a formal Indian legal document using the configured AI prompt.

8.8.5 Contract Review and Bulk Review

- Single contract review accepts text or uploaded file.
- Bulk review accepts 2– 20 contracts.
- Files are reviewed concurrently with asyncio.gather.
- Results provide batch statistics, safest/riskiest callouts, sorting and expandable per-file findings.
- Per-file failures are isolated in bulk processing.

Running reviews concurrently with asyncio.gather shortens the time needed for a batch, and isolating per-file failures means that one unreadable file does not abort the others.

8.8.6 Redline DOCX

- Supports text, cached review and uploaded-file redline flows.
- DOCX contains cover information, executive summary, contract text, risk annotations, missing protections, favourable points and recommendations.
- Risky clauses can be highlighted and annotated.

The redline document is generated entirely by the backend with python-docx, so the user receives an editable file that can be shared with a counterparty.

8.8.7 Legal Research

The research endpoint uses a legal-research system prompt and Gemini to produce structured research output. It is AI-assisted and should not be represented as an exhaustive authoritative legal database.

8.8.8 Fraud Detection

- Supports message, email and URL content.
- Stores scan history.
- Suspicious/dangerous results create masked community alerts.
- Seven dashboard submodules expose the fraud suite.

The seven fraud submodules in the dashboard share the same detection backend, which keeps the behaviour consistent across message, email and URL analysis.

8.8.9 SmartModule Engine

The backend contains a MODULE_SPECS registry. Each generic module defines title, fields, required inputs and an AI system prompt. The frontend requests a specification, renders a dynamic form, posts inputs to /api/module/run and displays the structured JSON response. This common architecture powers 21 generic modules.

This registry-driven design is the main reason the dashboard can expose 21 generic modules without 21 separate pages.

8.8.10 CNR Tracker

- Validates CNR input length.
- Decodes CNR components where supported.
- Checks official e-Courts portal reachability.
- Provides an official portal deep link.
- Provides AI-inferred stage/next-step information where live official status is not directly available.

The CNR tracker is honest about its limits: the decoded components and the deep link are deterministic, while the stage and next steps are labelled as AI-inferred.

8.8.11 Lawyer Directory

- Current source seeds 40 lawyer records.
- Supports city, specialisation, fee, experience and rating filters.
- Supports sorting and profile lookup.
- Creates consultation requests.
- No payment processing is currently implemented.

The directory is seeded project data and no payment is processed, so a consultation request is a recorded enquiry rather than a booking.

8.8.12 Contract Templates

The current source contains 12 templates: Mutual NDA, One-way NDA, Employment, Consultant, Rental, Commercial Lease, Freelance SoW, MoU, Founders' Agreement, Power of Attorney, Will and IP Assignment.

- Typed fields and defaults are presented in the UI.
- Gemini generates completed text.
- DOCX download is supported.
- Public share links are supported.
- Template text can be translated into Hindi, Tamil, Telugu, Bengali and Marathi.

Because the template text is generated rather than copied from a fixed file, the typed fields entered by the user shape the final document.

8.8.13 Voice and Multilingual Support

The Voice module produces spoken-style structured answers and localizes output labels/content for supported languages. The current page uses text input; browser speech recognition is not implemented. The template translation feature supports English, Hindi, Tamil, Telugu, Bengali and Marathi, while the generic Multi-Language module accepts a wider target-language list in its prompt.

8.9 Interface and Report Specifications

The user interface is built from a small number of reusable forms, filters and result views. This section describes the principal interface elements and the report types that the system produces.

8.9.1 Authentication Form

The authentication page is the first screen a visitor sees after the landing page. It validates the fields before the request is sent and displays the error returned by the backend when registration or login fails.

Field	Type	Validation
Name	Text	Required for registration
Email	Email	Required/valid
Password	Password	Minimum six characters in current backend

8.9.2 Chatbot

The chatbot screen is a conventional conversation view in which the answer grows as the stream arrives.

Element	Behaviour
Message	Natural-language legal query
Send	Starts SSE stream
History	Shows saved sessions/messages

8.9.3 Upload

The upload control is shared by the document analyzer and the contract review pages.

Element	Behaviour
File	PDF/DOCX/TXT upload
Analyze/Review	Runs AI workflow
Clear	Removes selected file

8.9.4 Fraud Form

The content type can be left on automatic so that the backend decides whether the submission is a message, an email or a URL.

Field	Purpose
Content Type	Message/email/URL/auto
Content	Suspicious material
Scan	Risk assessment

8.9.5 Lawyer Directory

The filters can be combined, and the results can be sorted using the supported options.

Filter	Purpose
City	City filter
Specialisation	Practice-area filter
Maximum Fee	Budget filter
Minimum Experience	Experience filter
Minimum Rating	Rating filter
Sort	Supported sort options

8.9.6 Template Library

The template page presents a typed field form so that the generated document reflects the values entered by the user.

Feature	Output
Search/Category	Template discovery
Field Form	Template inputs
AI Generate	Legal text
Download DOCX	Editable document
Translate	Regional-language text
Share	Public share view

8.9.7 Report Types

Every report is produced from a structured response and can therefore be displayed on screen, stored for later retrieval and, for the DOCX outputs, downloaded as a file.

Output	Content

Document Analysis	Key findings, obligations, risks, missing items
Contract Review	Risk score, red flags, missing protections, favourable points, recommendations
Bulk Report	Batch statistics and per-file details
Redline DOCX	Executive summary, contract body, annotations and recommendations
Fraud Result	Risk assessment and safety guidance
Research	AI-assisted legal research output
Petition Draft	Generated formal legal document
Template DOCX	Generated agreement/document
CNR Result	Decoded info, portal link, AI-inferred stage/next steps

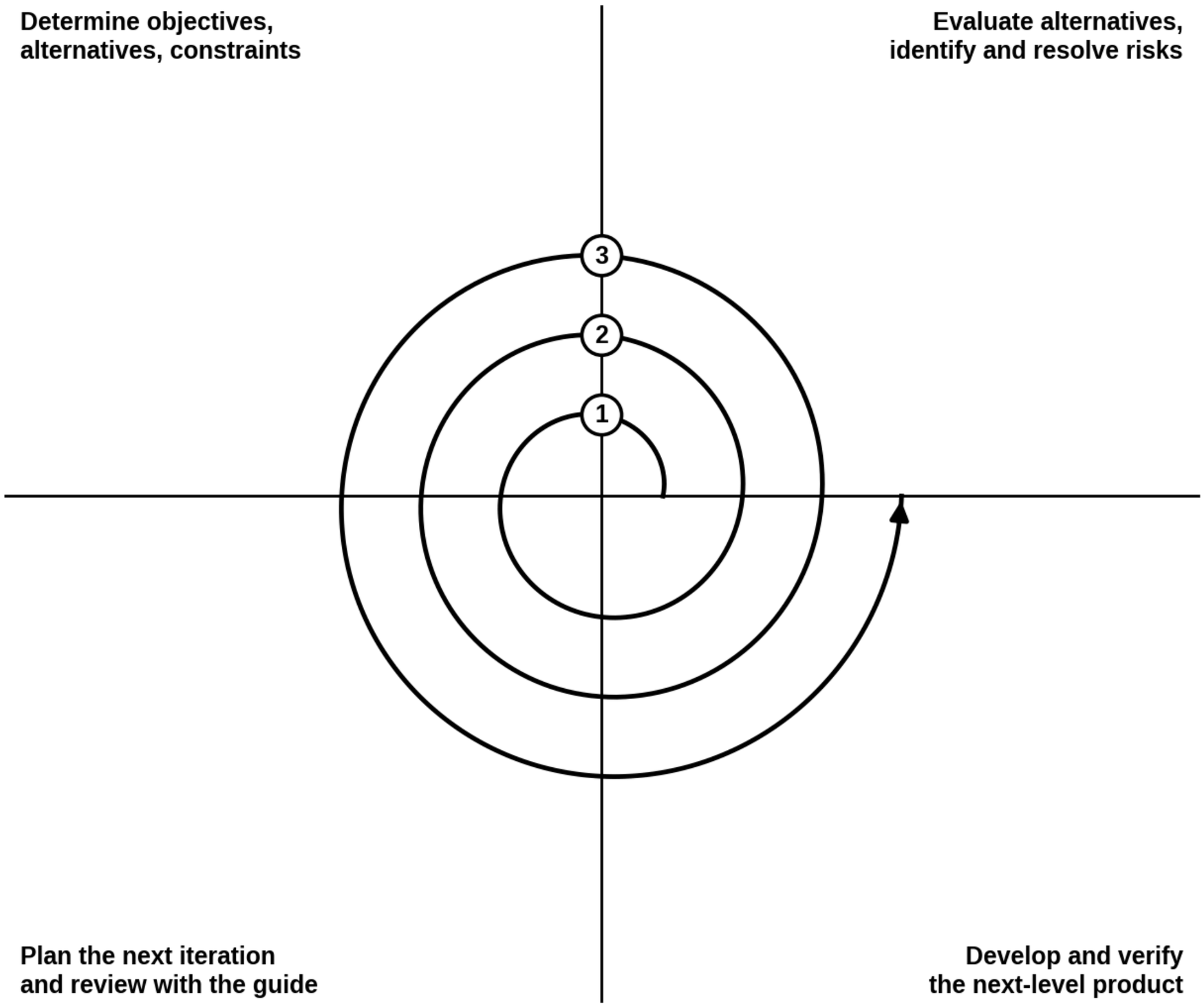
9. Spiral Model Iterations

This chapter describes how Nyaya Mitra was developed using the Spiral Model. The model was chosen because the project combines a large and evolving scope with a significant technical risk, namely its dependence on a language model whose output must be validated. The chapter explains the model, maps the recorded development increments to three spiral iterations and describes the planning, risk analysis, design refinement and validation work carried out in each.

9.1 Overview of the Spiral Model

The Spiral Model, proposed by Barry Boehm, is a risk-driven software process model. Development proceeds through repeated loops of a spiral, and each loop passes through four activities: determining objectives, alternatives and constraints; evaluating alternatives and identifying and resolving risks; developing and verifying the next-level product; and planning the next iteration. The radial distance from the centre represents cumulative cost, and the angular movement represents progress through the four activities. A prototype or working increment is produced at the end of every loop, and the decision to continue is based on the risks that remain.

The model suits Nyaya Mitra for three reasons. First, the project scope was not fixed at the start: the original 35-module concept grew into a 37-module dashboard with uploads, a CNR tracker, a lawyer directory, bulk review, redline output, a template library and regional-language translation. A model that expects requirements to be refined in successive loops accommodates this growth better than a strictly sequential model. Second, the main risks, such as incorrect AI output, privacy of legal documents and dependence on an external service, can be analysed explicitly at the start of each loop. Third, every loop delivers a working and testable system that the guide can review.



1 Iteration 1: Planning and Risk Analysis (baseline, first increment)

2 Iteration 2: Requirements and Design Refinement (uploads, CNR, lawyers, bulk review, redline, templates)

3 Iteration 3: Validation (translation, localisation, regression testing, release readiness)

Figure 9.1. Spiral model applied to Nyaya Mitra

The recorded development history of the project consists of six increments, each with its own deliverable and test report. For the purposes of this document the six increments are grouped into three spiral iterations, as shown below. The first iteration establishes the baseline, the second refines requirements and design as new workflows are added, and the third validates and localises the product.

Increment	Major deliverable	Spiral iteration
1	Authentication, 35-module baseline, six flagship modules, SmartModule engine and fraud feed	Iteration 1: Planning and Risk Analysis
2	File uploads, CNR tracker and lawyer directory with consultation	Iteration 2: Requirements and Design Refinement
3	Bulk contract review	Iteration 2: Requirements and Design Refinement
4	Redline DOCX and 12-template library	Iteration 2: Requirements and Design Refinement
5	Regional-language template translation	Iteration 3: Validation
6	Voice and Multi-Language localisation fixes and regression verification	Iteration 3: Validation

9.2 Iteration 1: Planning and Risk Analysis

9.2.1 Objectives and Planning

The objective of the first iteration was to decide whether the Nyaya Mitra idea was worth building and to produce a minimal but real foundation. The project identified the problem described in Chapter 3, defined the initial scope as a 35-module dashboard, selected the technology stack and decided that the system would be an assistance platform and not a source of binding legal advice. Constraints identified at this stage were the academic time frame, the absence of a licensed legal database and the need to use an external language model rather than training one.

Alternatives can be compared for each major decision. Training a custom model is impractical within an academic project, so a hosted LLM accessed through an integration library was chosen. A relational database would also have been possible, but MongoDB suits the structured AI results, which vary in shape from one workflow to another. A monolithic server-rendered application was set aside in favour of a React frontend with a FastAPI backend, which separates presentation from logic.

9.2.2 Technical Feasibility

The project uses mature web technologies and an external LLM service, making it feasible for an academic full-stack implementation without hosting a custom language model.

The technical risk is concentrated in a single dependency, the language model. By confining every call to one service layer in the backend, the design limits the effect of changes in that dependency to a small part of the code.

9.2.3 Economic Feasibility

Open-source frameworks keep development cost low. Main recurring costs for production would be LLM usage, hosting, database/storage and external service integrations.

For an academic project the recurring costs are negligible, because the frameworks are open source and the AI usage during development is small. For a public deployment the cost model would be dominated by language-model usage, which is another reason for the future rate-limiting and usage-monitoring enhancements.

9.2.4 Operational Feasibility

The unified dashboard, dedicated flagship pages, generic module renderer, document workflows, filters and downloadable outputs support a practical single-workspace experience.

Operational feasibility also depends on user trust. The interface therefore presents results in a structured form and displays a disclaimer that AI-generated output must be verified.

9.2.5 Risk Analysis

The principal risks identified in the first iteration are listed below, together with a qualitative rating of likelihood and impact assigned for this document and the mitigation adopted in the design.

ID	Risk	Likelihood	Impact	Mitigation
R1	Incorrect or fabricated legal information from the language model	High	High	Structured prompts, visible disclaimers, AI output labelled as AI-generated, advice to verify with a professional
R2	Exposure of sensitive documents and histories	Medium	High	Authentication required, histories scoped to the user, no direct browser-to-LLM calls, hashed passwords
R3	Dependence on an external LLM service for availability and cost	Medium	High	Single service layer so the provider can be replaced; controlled error responses
R4	Official court data not available because of CAPTCHA or API limits	High	Medium	Deep link and reachability check; stage clearly labelled as AI-inferred
R5	Scope growth across many modules	High	Medium	Shared SmartModule engine, six dedicated flagship pages, incremental delivery
R6	Unauthorised access to protected operations	Low	High	bcrypt hashing, JWT bearer validation, authorisation tests
R7	Invalid or oversized file uploads	Medium	Medium	File type and size validation; controlled validation errors

9.2.6 Prototype Developed

The first increment delivered registration and login with JWT, the dashboard backed by a registry of 35 modules, the six flagship workflows (chatbot, document analyzer, petition and notice drafting, contract review, legal research and fraud detection), the SmartModule engine that serves the generic modules and the community fraud alert feed. Although the module count was later increased to 37, the structure created in this iteration remained the foundation of the later work.

9.2.7 Evaluation and Plan for the Next Iteration

The recorded test result for this increment is 21 of 21 backend tests passed, with the six flagship and baseline dashboard flows verified end to end. On the basis of this result and of a review of the prototype, the next iteration was planned around practical needs that the baseline did not yet satisfy: reading documents from files, assisting with court case numbers, connecting users with lawyers, comparing several contracts and producing editable output.

9.3 Iteration 2: Requirements and Design Refinement

9.3.1 Objectives

The second iteration refined the requirements in the light of the prototype and extended the design to support them. Its objective was to turn the baseline into a more practical workspace without destroying the simple structure created earlier. Three increments were completed in this iteration, covering uploads, the CNR tracker and the lawyer directory (increment 2), bulk contract review (increment 3) and redline output with the template library (increment 4).

9.3.2 Requirement Refinement

A review of the prototype showed that copying and pasting long documents is inconvenient, so requirements FR-07 and FR-16 were added to support PDF, DOCX and TXT uploads and the review of 2 to 20 contracts at once. Court-related questions led to FR-14, the CNR assistance requirement, which was written to promise assistance and an official link instead of guaranteed live status, because of the access limits identified as risk R4. FR-15 added the lawyer directory so that the preparation of a document can lead to a consultation.

Later in the iteration, requirements FR-17 to FR-20 introduced the redline document, the 12-template library, DOCX download and public sharing. As a result of these changes the dashboard grew to 37 modules and the data model grew to 13 collections. The refined requirements are the ones presented in Chapter 6, and the use cases of Chapter 7 were extended accordingly.

9.3.3 Module Inventory

The refined system exposes the 37 modules listed below. The first 19 are the Core Legal Services, the next 11 are the Advanced AI Features and the last 7 form the AI Scam and Fraud Detection suite.

Category	No.	Module
Core Legal Services	1	Legal Case Management
Core Legal Services	2	AI Document Analyzer
Core Legal Services	3	Online Legal Consultation
Core Legal Services	4	Court Case Tracking
Core Legal Services	5	Contract Review Automation
Core Legal Services	6	Legal Research Engine
Core Legal Services	7	Cyber Crime Reporting

Category	No.	Module
Core Legal Services	8	Intellectual Property Portal
Core Legal Services	9	E-Governance Complaint
Core Legal Services	10	Consumer Grievance Platform
Core Legal Services	11	Legal Knowledge Chatbot
Core Legal Services	12	Compliance Monitoring
Core Legal Services	13	Digital Evidence Vault
Core Legal Services	14	Online Dispute Resolution
Core Legal Services	15	Legal Risk Assessment
Core Legal Services	16	Traffic Violation Manager
Core Legal Services	17	RTI Application Tracker
Core Legal Services	18	E-Notary Management
Core Legal Services	19	Judicial Data Analytics
Advanced AI Features	1	AI Legal Research Assistant
Advanced AI Features	2	Petition&Notice Drafting
Advanced AI Features	3	AI Contract Review Agent
Advanced AI Features	4	Bulk Contract Review
Advanced AI Features	5	Contract Templates Library
Advanced AI Features	6	Legal Client CRM
Advanced AI Features	7	Court Case Tracking Agent
Advanced AI Features	8	Voice-Based Legal Assistant
Advanced AI Features	9	Multi-Language Legal Support
Advanced AI Features	10	Mock Court (Law Students)
Advanced AI Features	11	Lawyer Marketplace
AI Scam&Fraud Detection	1	Message Scam Detection
AI Scam&Fraud Detection	2	Email Fraud Detection
AI Scam&Fraud Detection	3	Phishing URL Detection
AI Scam&Fraud Detection	4	Real-Time Risk Scoring
AI Scam&Fraud Detection	5	Fraud Reporting System
AI Scam&Fraud Detection	6	Community Threat Alerts
AI Scam&Fraud Detection	7	Cyber Awareness Assistant

9.3.4 Design Refinement

The most important design decision of this iteration was to keep the SmartModule engine as the default way of adding a module. A module is defined by a title, a list of fields, the required inputs and an AI system prompt in the MODULE_SPECS registry. The frontend asks for the specification, renders a form, posts the inputs and displays the structured JSON result. Because of this approach, 21 modules are served without 21 dedicated pages, and the maintenance cost of the dashboard remains low.

A second decision concerned concurrency. Bulk review had to process up to 20 files in a single request, so the backend runs the reviews concurrently with asyncio.gather and isolates failures so that a single unreadable file does not abort the batch. A third decision concerned output: the redline document and the template DOCX are generated on the server with python-docx, giving the user an editable Word file. The resulting architecture, data model and interfaces are documented in Chapter 8.

9.3.5 Risk Re-assessment

As new capabilities were added the risk analysis was repeated. The risks that became more important in this iteration are listed below.

ID	Risk	Mitigation
R8	Slow or failing requests when many contracts are reviewed together	Concurrent execution with per-file failure isolation; limit of 2 to 20 files
R9	Generated DOCX files that cannot be opened	Generated files verified as valid python-docx documents in recorded tests
R10	Template text mistaken for professionally verified legal drafting	Templates are labelled as AI-generated drafts to be reviewed before use
R11	Mistakes in translation that alter legal meaning	Protected fields preserved; translated output verified for script and proper nouns
R12	Seeded lawyer records mistaken for verified professionals	Directory described as seeded project data; verified onboarding listed as a future enhancement

9.3.6 Increments Delivered and Evaluation

The recorded results for the increments of this iteration are as follows. Increment 2 added 26 backend tests, all of which passed, and verified the upload, CNR and lawyer workflows. Increment 3 passed 7 of 7 backend tests and verified the bulk review end to end. Increment 4 passed 17 of 17 backend tests and verified templates, redline generation, DOCX output and sharing. At the end of the iteration the system had all of the workflows described in this document except the regional-language translation and the final localisation fixes, which were planned for the validation iteration.

9.4 Iteration 3: Validation

9.4.1 Objectives

The third iteration concentrated on validation. Its objectives were to add regional-language translation for templates (increment 5), to correct the localisation of the Voice and Multi-Language modules and verify the whole system by regression testing (increment 6), and to review security, privacy and legal-safety requirements before presenting the system for evaluation.

9.4.2 Test Strategy

The supplied project archive contains backend pytest suites and iteration-specific end-to-end test reports. Validation covers authentication, module registry, AI workflows, uploads, CNR tracking, lawyer filters and consultation, bulk review, redline generation, templates, sharing, translation and multilingual output.

Level	Coverage
Backend/Unit	Validation, authentication, endpoint logic, file extraction, module registry and document generation.
Integration	FastAPI– MongoDB and AI workflow integration.
End-to-End	Landing/auth/dashboard/flagship/SmartModule/upload/CNR/lawyer/bulk/template/multilingual flows.
Authorization	Missing/invalid auth and ownership guards.
Document	Generated DOCX files verified as valid python-docx documents in recorded tests.

The strategy combines automated and end-to-end testing. Backend tests exercise the endpoints and the supporting functions directly, while the end-to-end tests drive the user interface through the main flows. The authorisation tests verify that missing or invalid tokens are rejected and that ownership checks prevent access to another user's records.

9.4.3 Recorded Test Results

Iteration	Recorded Result
1	21/21 backend tests passed; six flagship and baseline dashboard flows verified.
2	26/26 new backend tests passed; upload, CNR and lawyer workflows verified.
3	7/7 backend tests passed; bulk review E2E verified.
4	17/17 backend tests passed; templates, redline, DOCX and sharing verified.
5	13/13 relevant backend tests passed; five non-English template translations and caching verified.
6	11 tests passed, 1 expected skip, 0 failures; Voice/Multi-Language localization verified.

The results of the last two increments belong to this iteration. Increment 5 passed 13 of 13 relevant backend tests and verified five non-English template translations and their caching. Increment 6 produced 11 passing tests with one expected skip and no failures, and verified the localisation of the Voice and Multi-Language modules. The earlier results, from increments 1 to 4, are retained in the table because they form the regression baseline that increment 6 had to preserve.

9.4.4 Representative Test Cases

The table lists representative test cases that cover authentication, the AI workflows, file handling, CNR, lawyer filters, templates, sharing, translation and authorisation. They are traceable to the requirements in Chapter 6 and the use cases in Chapter 7.

ID	Scenario	Expected Result
TC-01	Valid registration	Account/JWT returned
TC-02	Wrong password	Authentication denied
TC-03	Chatbot query	Streaming response and saved session
TC-04	Valid upload	Text extracted and analysed
TC-05	Invalid/large file	Controlled validation error
TC-06	Contract review	Structured risk result
TC-07	Bulk review	Batch statistics and league table
TC-08	Redline	Valid DOCX download
TC-09	CNR	Validation/decoding and official link
TC-10	Lawyer filter	Filtered records
TC-11	Template generation	Generated legal text
TC-12	Public share	Shared page accessible
TC-13	Hindi translation	Hindi script and preserved proper nouns
TC-14	Unauthorized API	401/authorization error

Each test case states a scenario and an expected result that can be observed without ambiguity. For instance, TC-02 expects authentication to be denied when the password is wrong, TC-05 expects a controlled validation error for an invalid or large file, TC-08 expects a valid downloadable DOCX from the redline endpoint and TC-14 expects an authorisation error when an API is called without a valid token.

9.4.5 Security, Privacy and Legal-Safety Requirements

Security and privacy requirements are particularly important for a legal-technology platform because users may submit confidential agreements and sensitive facts. The following controls were reviewed during validation.

- Never store plaintext passwords.
- Require JWT for protected operations.
- Scope user histories to authenticated identity.
- Expose only intentionally shared content publicly.
- Validate uploaded file type and size.
- Use restrictive CORS in production.
- Rate-limit expensive LLM endpoints in production.
- Clearly distinguish AI-inferred information from official government/court data.
- Display legal-information disclaimers and avoid representing AI output as guaranteed legal advice.

Several of these controls are fully implemented in the current build, namely password hashing, JWT protection, user-scoped histories, restricted public exposure and file validation. Others, such as restrictive CORS and rate limiting of expensive LLM endpoints, are production recommendations that apply when the system is deployed publicly. The last two controls, the distinction between AI-inferred and official information and the display of legal disclaimers, are legal-safety measures and are visible in the interface and in the documentation.

9.4.6 Current Limitations

Honest statement of limitations is part of validation. The current build has the following limitations, which should be considered when the system is evaluated.

- No stored formal user roles/RBAC.
- CNR stage may be AI-inferred rather than live official status.
- Research is LLM-assisted rather than a maintained authoritative RAG database.
- Voice input is not yet browser-speech enabled.
- Lawyer records are seeded project data rather than live professional verification.
- No payment gateway.
- No legally binding e-signature/e-notary.
- AI output can contain errors and requires verification.

These limitations do not prevent the system from demonstrating its purpose, but they define where users must apply caution. They also provide the starting point for the future enhancements listed in Chapter 10.

9.4.7 Evaluation and Release Readiness

At the end of the third iteration, all recorded test suites passed with no failures, the 37-module dashboard and the additional workflows were verified, and the documentation was aligned with the actual source code. The system is therefore considered ready for academic evaluation and demonstration. It is not presented as production-ready: formal role-based access control, authoritative legal retrieval, official court-data integration, payments and production security hardening remain open items for a further iteration of the spiral.

9.5 Summary of the Spiral Iterations

Iteration	Principal objective	Key activities	Outcome
1. Planning and Risk Analysis	Decide feasibility and create the baseline	Scope definition, stack selection, feasibility study, risk analysis R1-R7, baseline prototype	Baseline with six flagship modules and SmartModule engine; 21/21 tests passed
2. Requirements and Design Refinement	Refine requirements and extend the design	Uploads, CNR, lawyer directory, bulk review, redline, templates; risk re-assessment R8-R12	37-module dashboard, 13 collections; 26/26, 7/7 and 17/17 tests passed
3. Validation	Verify, localise and review	Translation, localisation fixes, regression testing, security and limitations review	All suites passed; ready for academic evaluation

The spiral approach allowed the project to learn from each loop. Risk analysis at the beginning of every iteration influenced the requirements and design choices that followed, and every iteration produced a working system that could be tested and demonstrated.

10. Conclusion and Future Enhancements

10.1 Conclusion

Nyaya Mitra has evolved beyond the original 35-module concept into a 37-module implemented dashboard with a broader set of AI, document and legal-service workflows. The current source demonstrates a complete academic full-stack architecture using React, FastAPI, MongoDB, JWT, Gemini 3 Flash, document-processing libraries and structured module execution.

The most important SRS correction is to distinguish implemented functionality from roadmap items. The current build supports the 37-module dashboard and the additional workflows documented here, while formal RBAC, authoritative live government-data integration, payments, browser voice input and production-grade legal retrieval remain future enhancements. This makes the SRS more accurate, testable and suitable for academic evaluation.

The development of Nyaya Mitra shows that a broad legal-assistance workspace can be built with mainstream tools. The React frontend, FastAPI backend and MongoDB database provide a maintainable foundation, the language model supplies the language understanding, and libraries such as pypdf and python-docx connect the system with the documents that users actually have. The shared SmartModule engine demonstrates that a large number of modules can be supported by a small amount of code, and the structured-output approach makes the results easy to display, store and download.

The project also carries lessons about responsible use of AI in a sensitive domain. The system consistently describes itself as an assistance platform, labels AI-inferred information, avoids guaranteeing court data that it cannot obtain, and records its limitations openly. These choices are as important as the technical features, because the value of a legal-technology tool depends on the trust that users can place in its output.

From the point of view of the academic objectives, the project has exercised the complete life cycle: problem analysis, feasibility study, requirements specification, design with DFD, UML and ER models, spiral development in three iterations, implementation, testing and documentation. The result is a documented, testable and demonstrable system whose scope and limitations are stated accurately.

10.2 Future Enhancements

The enhancements below are grouped by theme. They are not part of the current release, and several of them depend on external agreements or infrastructure.

Access control and security. The first priority is a formal role model with Client, Lawyer, Law Student and Administrator roles, so that authorisation is based on stored roles and not on the workflows the user chooses. Secure account recovery and profile management should accompany it. For a public deployment,

production rate limiting, audit logging, strict CORS configuration, secret rotation and security monitoring are needed, and the FastAPI server should be divided into routers and services to keep the code maintainable.

Quality of legal content. Legal research would become more trustworthy if it were grounded on an authoritative retrieval layer with source and version tracking instead of relying on the language model alone. Official e-Courts and CNR integration, where it is permitted, would replace the AI-inferred stage with verified status. Advanced contract version comparison would extend the current redline and bulk review.

Reach and convenience. Browser speech input and text-to-speech would turn the voice module into a true voice assistant, and more regional Indian languages could be supported across all modules. Native Android and iOS applications, a browser phishing extension and a mobile SMS and email companion would bring the platform and its fraud detection to the places where users receive suspicious content. A saved-draft and history browsing interface would make the stored records easier to use.

Marketplace and business features. Verified lawyer onboarding with stronger credential checks would address the limitation that directory records are seeded data, and payment or subscription integration would allow consultations to be booked and paid for within the platform.

Summary of the roadmap:

- Formal Client/Lawyer/Law Student/Admin RBAC.
- Secure account recovery/profile management.
- Authoritative legal RAG with source/version tracking.
- Official e-Courts/CNR API integration where permitted.
- Browser speech input and text-to-speech.
- More regional Indian languages across all modules.
- Android/iOS applications.
- Payment/subscription integration.
- Production rate limiting, audit logging, strict CORS, secret rotation and security monitoring.
- Browser phishing extension and mobile SMS/email companion.
- Saved draft/history browsing UI.
- Split FastAPI server into routers/services.
- Verified lawyer onboarding and stronger credential checks.
- Advanced contract version comparison and redlining.

11. Bibliography

The following references were used in the design and development of the system and in the preparation of this document.

- React Documentation — https://react.dev
- React Router Documentation — https://reactrouter.com
- FastAPI Documentation — https://fastapi.tiangolo.com
- MongoDB Documentation — https://www.mongodb.com/docs
- Google AI / Gemini API Documentation — https://ai.google.dev
- PyJWT / JSON Web Token documentation
- python-docx documentation
- pypdf documentation
- India Code — https://www.indiacode.nic.in
- e-Courts resources — https://ecourts.gov.in
- RTI Online — https://rtionline.gov.in
- Pressman, R. S., Software Engineering: A Practitioner’ s Approach.
- Sommerville, I., Software Engineering.

Source alignment note: This document was revised using the supplied Smart Legal Advisor SRS and the supplied NyayMitra project archive, including its backend API definitions, frontend routes, module registry, PRD and recorded test reports.

Additional references on software process, modelling, security and applicable Indian law:

- Boehm, B. W., 'A Spiral Model of Software Development and Enhancement', IEEE Computer, vol. 21, no. 5, 1988.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Booch, G., Rumbaugh, J. and Jacobson, I., The Unified Modeling Language User Guide.
- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems.
- Jones, M., Bradley, J. and Sakimura, N., RFC 7519: JSON Web Token (JWT), IETF, 2015.
- The Indian Contract Act, 1872 –- https://www.indiacode.nic.in
- The Information Technology Act, 2000 -– https://www.indiacode.nic.in
- The Digital Personal Data Protection Act, 2023 –- https://www.indiacode.nic.in